מכללת אורט הרמלין, נתניה
תשע"ז, 2017

עבודת גמר – טכנאי תוכנה

אבלון

מנחה: מיכאל צ'רנובלסקי וזהבה יעקבסון
שם מגיש: בורודקין מריאל
ת"ז: 208584383

תוכן עניינים

מבוא.....2
מטרה.....2
תיאור משחק.....2
חוקי משחק.....3
שפת תכנות ופירוט סביבת העבודה והכלים.....4
נתוני מחשב.....4

מפרטי תוכנה.....5
הצגת תחילת המשחק.....5
הנחות בניית המשחק.....8

הסתכלות על הבנייה בקוד **9**
מטריצת כיוונים 9
מטריצת בניית לוח 9

מחלקות ומבנה נתונים **11**
המחלקות 11
פירוט תמציתי של המחלקות 11
שימוש במערך-רשימות (ArrayList) 12

אלגוריתם תפקוד הלוח והכלים **13**
בדיקות סימון חוקי של כדורים 13
ביצוע לוחמת הזזה 15
מיון כדורים 17
הזזת כדורים 19
בדיקת הוצאת כדור וניצחון 20

בינה מלכותית **20**
הבחנה: הורשת של אנוש ומחשב משחקן 20
מחלקה אבסטרקטית 'שחקן' 21
מחלקת 'שחקן מחשב' 21
מחלקת 'מהלך' 23
מחלקת 'כתם' 25
חשיבת מינ-מקס 27

ניספח – קוד התוכנית **29**

מבוא

מטרה

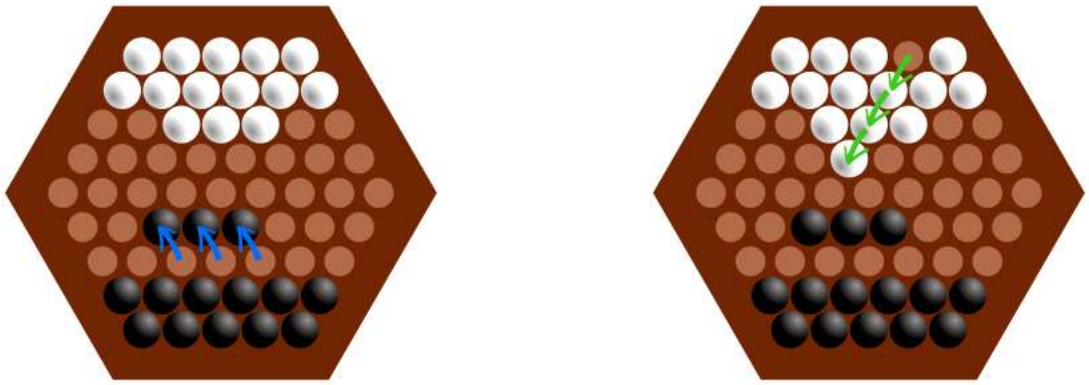
כשניגשתי לבחור רעיון, הייתי סגור על זה שברצוני ללמוד על ראש המחשב ודרכי החשיבה שלו, ולבחון את כישוריו בתנאים לא לגמרי סטנדרטים. אהבתי את הרעיון של חשיבה שתתבצע בסביבה אלכסונית שתדרוש יחס שונה בחישוב הכיוונים, ותצריך לוגיקה מעט.. הפוכה.

בעת תחילתי לא זלזלתי במשחק ובאופן כללי נתן להגיד שהערכתי את רמת הקושי. עם זאת, רק בעבודה מעשית הבנתי כמה משחק זה מורכב ומורחב ביסודו, ולמדתי להשתמש בעקרונות תכנות מומחה עצמים על מנת להגיע למוצר יעיל וחזק.

תיאור המשחק

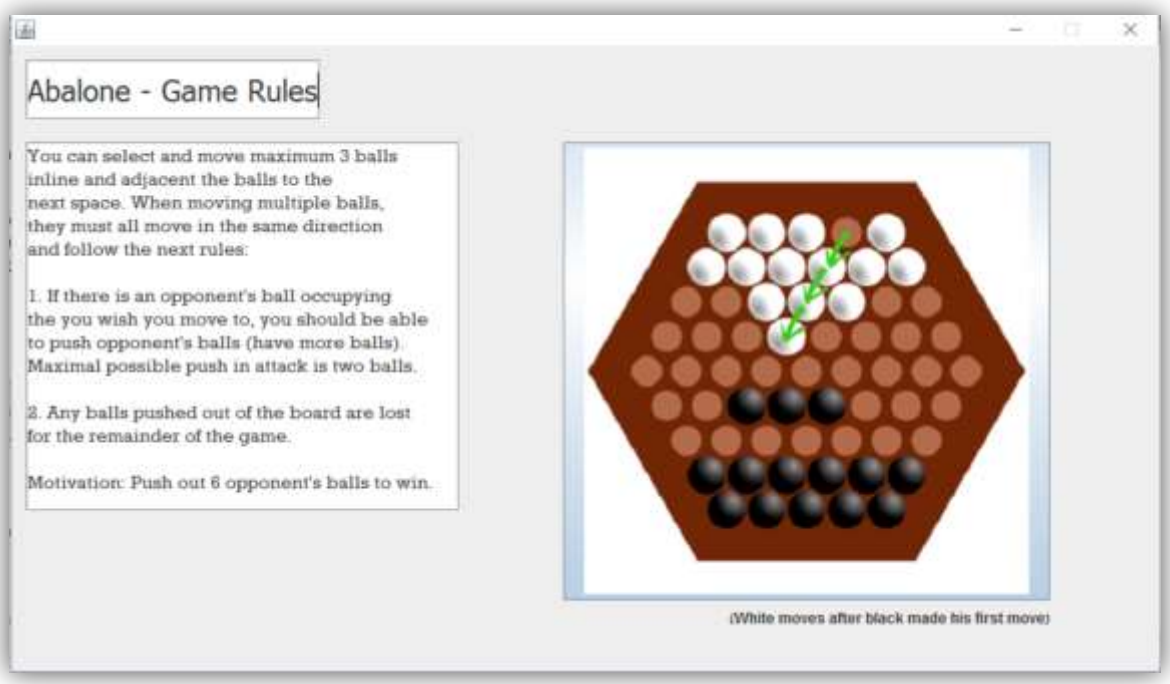
במשחק זה ניתן לשחק כ-2 משתמשים, או 1 כנגד המחשב.

המשחק בנוי מלוח אלבון שמורכב מ61 חורים בצורת משושה ו14 כדורים לכל צד, ואת המשחק התחיל הצד השחור. מטרת המשחק היא להוביל התקפות מנצחות שכוללות כמה שיותר כדורים כנגד היריב, ואם בשורה/טור הנתקף יש יותר כדורים מכדורי השחקן ממול, הכדורים יזיזו את כדורי האוייב (3 יכול להזיז 2 או 1, 2 יכול להזיז 1) השחקן מעבד כדור ברגע שנדחף אל מחוץ לאזור הלוח. המשחק נשאר בכוח ואפשרי לשני השחקנים בכל עת. במהלך המשחק השחקנים יתמודדו זה מול זה ויזיזו כל פעם כדור בצבעם: דוגמה להזזות בלי אפשריות:



(דוגמה לביצוע שתי מהלכים במשחק, כשהשחור מתחיל אותו והלבן משחק את הצעד הבא)

ולהסבר נוסף, ניתן בכל רגע להפעיל את ההסבר הקצר בזמן פעולת התוכנה באמצעות פתיחת פריים נוסף שמסביר את ההוראות בלחיצה על הסבר המוצג בתפריט למעלה:

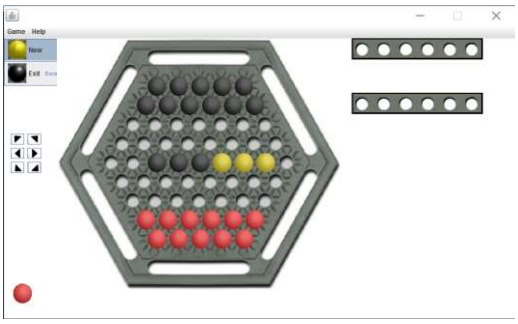


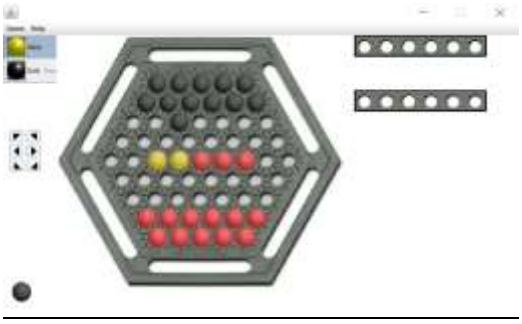
חוקי המשחק:

<u>מהלך המשחק:</u> השחקנים יתמודדו זה מול זה ויזוזו כל פעם כדור/כמה כדורים בצבע שלהם:

- לשחקן מותר להזיז עד 3 כדורים שלו בכל עת, ואי אפשר להזיז יותר מ3 כדורים של היריב.

- לשחקן מותר להזיז יחד שלושה כדורים בטור ובשורה במידע ואין כדור המונע תזוזה מהכדור.





(דוגמה לשתי הזזות לא חוקיות)

הכרעה: הניצחון מוענק לשחקן שנגמרו לו הכדורים על הלוח, ואילו תקו יוכרז במידע ולשני הצדדים נשאר כדור בודד.

שפת תכנות ופירוט סביבת העבודה והכלים:

פיתוח המערכת נעשה בשפת:JAVA

תוך כדי שימוש ב:NETBEANS IDE 8.2

נתוני מחשב:

ASUS K401U

CPU: Intel Core i7

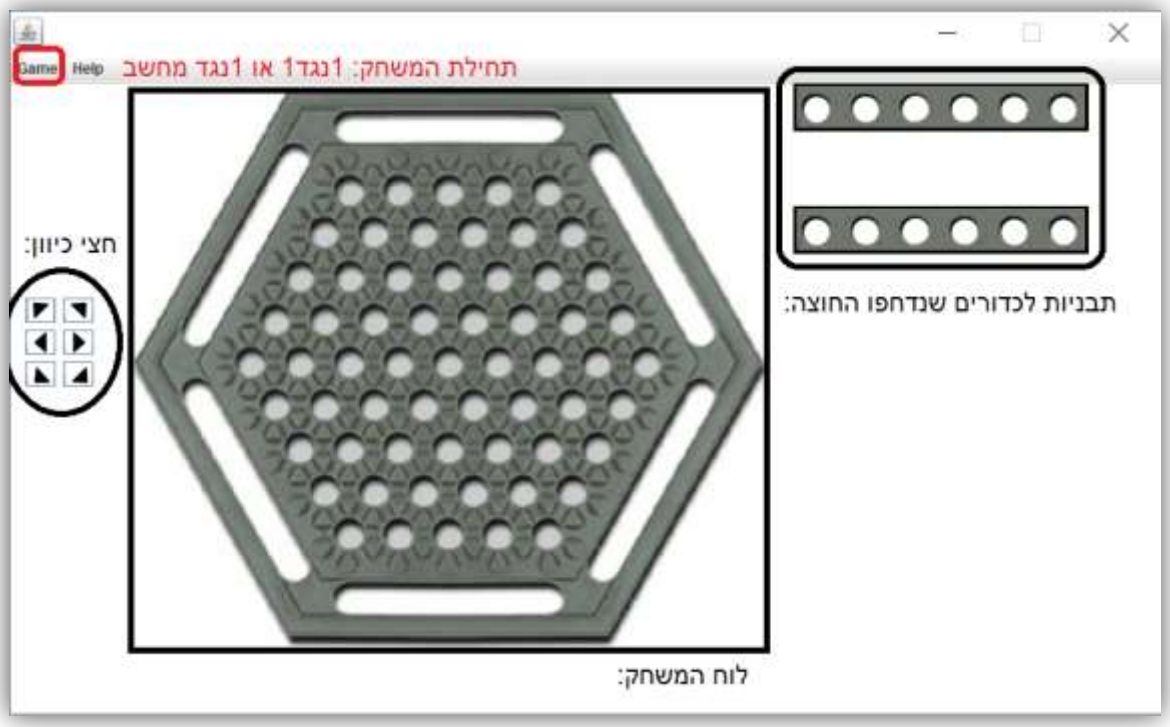
Memory: 16GB

OS: WIN10

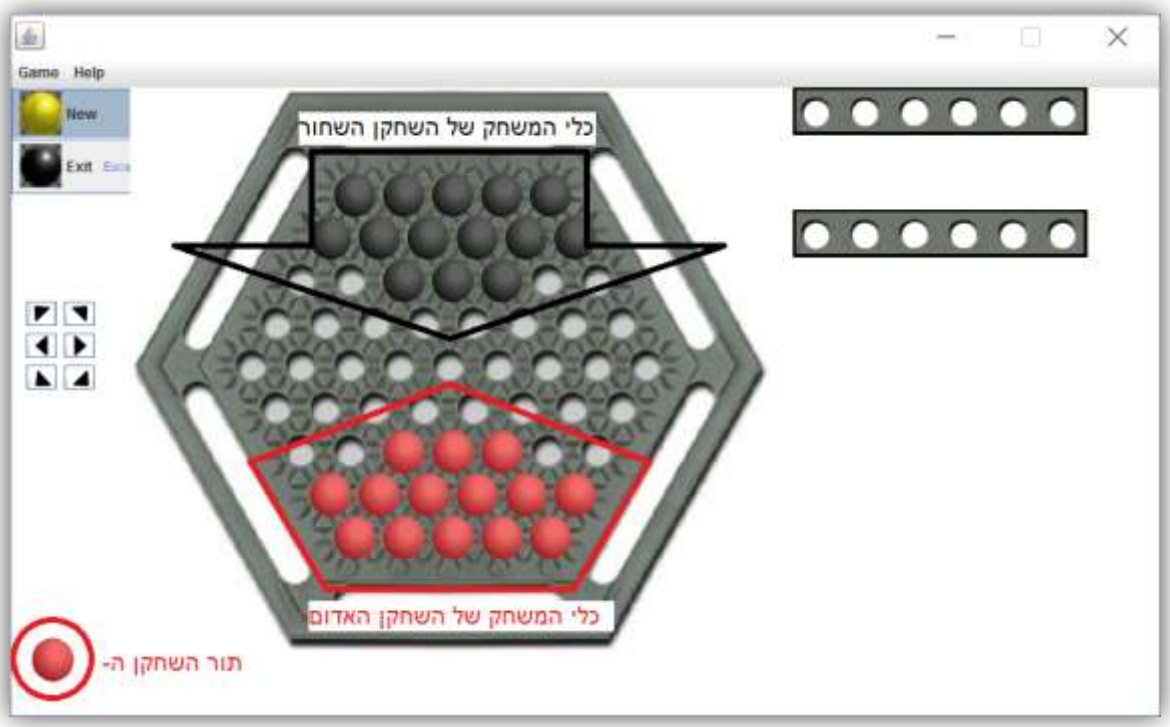
HDD: 1TB+24G(SSD)

מפרטי תוכנה

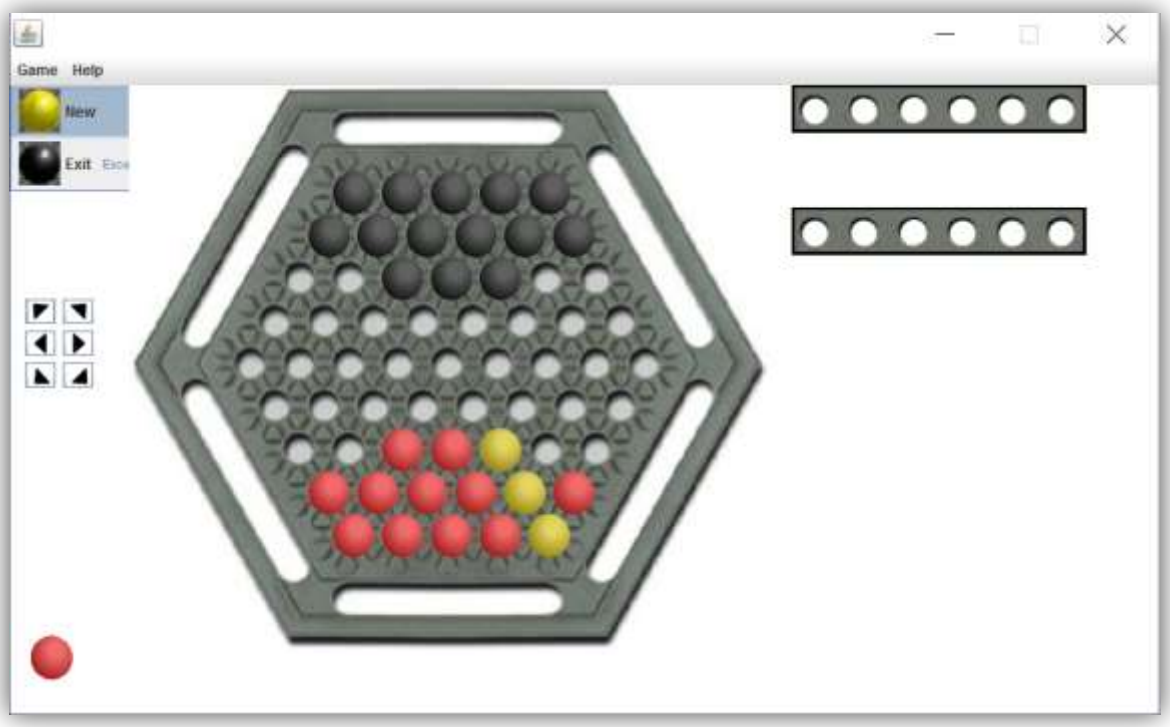
הצגת תחילת המערכת:



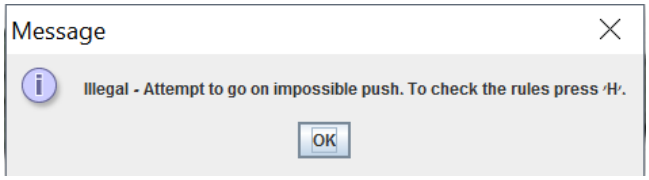
1. תחילת הרצת המשחק, בתפריט משחק ניתן לבחור תחילת משחקו, אלו הכלים ההתחלתיים לפני תחילת המשחק.

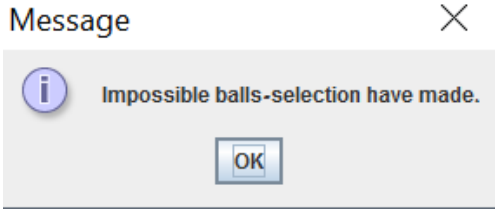


2. בעת תחילת המשחק (נגד שחקן או נגד מחשב) כל צד מקבל את כלי המשחק שלו. יש לציין שהאדומים נמצאים למטה והשחורים למעלה. האדומים מתחילים את המשחק. בצידו השמאלי למטה של המשחק ניתן להבחין בכדור שמציג את תור השחקן.



3. סימון יחשב חוקי במידע ומסמנים עד כ3 כדורים ברצף של שורה/אלכסון. סימונים אחרים לא יתאפשרו:





הנחות בניית המשחק:

ההנחה שאני עושה במשחק זה היא שהמשחק מתבצע על לוח אחד, בין אם הוא בין שחקנים או בין שחקן ומחשב. לכן, ניתן לרכז את אפשרויות הבחירה בתפריט המשחק, בהנחה שלוקחים בחשבון את האפשרות שהמשתמש אינו יודע כיצד משחק הולך ולכן יש לו בתפריט גם כן את האפשרות להיעזר במדריך הקצר למשחק בסיסי.

נאלצתי להשתמש בחישובים גרפים כדי שהציור יהיה מדוייק ונכון, הבנתי שלנוחיותו של המשתמש, כדי שיהיה לו בנוסף למשחק גם דרך להבחין בין תורות (בכדי שלא יצטרך לבדוק אילו כדורים כן ולא מסומנים לו), ולדעת את מצבו ומרחקו מהפסד.

בנוסף לכך, חשוב שלמשתמש תיהיה דרך נוחה לסמן/להוריד סימון של כדורים.

<u>בעיה אלגוריתמית 1:</u> תחילה תהיתי כיצד תיהיה דרכי הנכונה ביותר לסימון. מחקרים שעשיתי בנושא נטעו לסימון והסרת סימון בעזרת לחיצה. באלגוריתם סימון הראשון שלי אני השתמשתי בכפתור הסרת סימון אך בסופו של דבר, מטעמי נוחות וסדר, הבנתי שעדיפה פעולת סימון שתבדוק את סוג הכדור בכל לחיצה ובמידה ותור השחקן תואם את הכדור הנלחץ, כדור זה ישנה את הסטטוס שלו (מסומן/לא מסומן).

<u>בעיה אלגוריתמית 2:</u> סימון לא חוקי מצד המשתמש. חשוב להבהיר את החוקים, כדי להבטיח שלא תיהיה חפיפה. לכן היה צורך בבדיקה:

1. תבחין בין לחיצה על כדור שזה תורו ולבין כדור שלא, ותתעלם מכדורים שלא תורם.

2. תזהה את כל הכדורים כדי לבדוק רציפות שורתית בינהם + סוג של שני כדורים, ושל שלוש כדורים. צבעם של הכדורים משתנה כל עוד לא סומנו יותר מ-3 כדורים והחוקים מתבצעים.

הסתכלות על הבנייה בקוד

מטריצת כיוונים:

המשחק מתבסס לכל אורכו על צעדים שמתבצעים. וכמו שצויין לעיל, הלוח אינו רגיל אלא מורכב בצורה דיאגנלית, מה שצריך התייחסות שונה כלפיו. להלן מטריצת הכיוונים כפי שהיא מאותחלת בתוכנה:

```
/* Possible directions array to apply ball moves based on the selected direction */
static int[][] Direction
        = {// dx dy
            {-1, -1}, // LEFTUP
            {1, -1}, // RIGHTUP
            {2, 0}, // RIGHT
            {1, 1}, // RIGHTDOWN
            {-1, 1}, // LEFTDOWN
            {-2, 0} // LEFT
        };
```

מטריצת בניית לוח:

הלוח שנבנה צריך להתייחס לצורתיו, שבעצם בניוה בצורה דיאגנלית, מה שדורש התייחסות שונה לגבולותיו ולכדוריו כאחד. ידועות שיטות בנייה שונות בשוק המקצועי המתבססות על מחלקת הלוח עצמו אך לא ראיתי בכך צורך כיוון שאתחול הלוח מתבצע רק בתחילת משחק ולכן בחרתי להדגים את תמונת הלוח האתחול בעזרת שימוש של מטריצת ביטים דו מימדית:

```java
/* InitBoard for the board the painting */
public static byte[][] InitBoard = {
    {0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0},
    {0, 0, 0, 0, 0, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 0, 0, 0, 0, 0},
    {0, 0, 0, 0, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 0, 0, 0, 0},
    {0, 0, 0, 0, 1, 0, 1, 0, 2, 0, 2, 0, 2, 0, 1, 0, 1, 0, 0, 0, 0},
    {0, 0, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 0, 0},
    {0, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 0},
    {0, 0, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 0, 0},
    {0, 0, 0, 0, 1, 0, 1, 0, 3, 0, 3, 0, 3, 0, 1, 0, 1, 0, 0, 0, 0},
    {0, 0, 0, 0, 0, 3, 0, 3, 0, 3, 0, 3, 0, 3, 0, 3, 0, 0, 0, 0, 0},
    {0, 0, 0, 0, 0, 0, 3, 0, 3, 0, 3, 0, 3, 0, 3, 0, 0, 0, 0, 0, 0},
    {0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0}
};
```

ולמען חישובי גדלים אתחלתי <u>משתנים גלובים לשורה ועמודה:</u>

```java
/* Sizes of Rows and Cols in the board. */
public static final byte ROWS = 11;
public static final byte COLS = 21;
```

ולכלים הגדרתי תמונות שנמצאות בתיקיית הפרוייקט ובזמן הפעלת הבניי, מאותחלות:

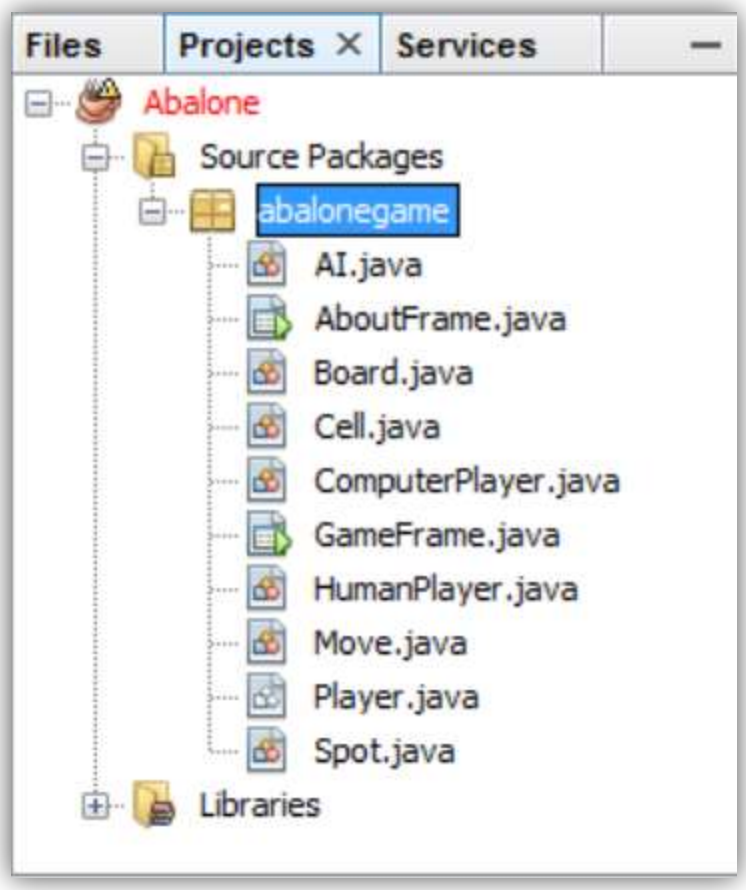
```java
/* Images */
public static Image RedBallImage, BlackBallImage, SelectedBallImage;

public Board(GameFrame Parent, Point Start, int Num) {

    /* Setting the images to to know to each image we relate */
    RedBallImage = Parent.getToolkit().createImage("Red.png");
    BlackBallImage = Parent.getToolkit().createImage("Black.png");
    SelectedBallImage = Parent.getToolkit().createImage("Selected.png");
```

(יש לקחת בחשבון שהמוצג לא זהה בקוד, אלא חיבור של שתי תמונות ממקומות קצת שונים להמחשת העיקרון)

מחלקות ומבנה נתונים

המחלקות:

להלן המחלקות של הפרוייקט שיוסברו במפורט בהמשך תיק הפרוייקט הנ"ל:



פירוט תמציתי של המחלקות:

AI - מחלקה היורשת מסראד ומתפעלת את הפעולות כדי להבטיח שלא תיהיה כפילות מהלכים באותו הזמן.

AboutFrame - פריים עזר המיועד לשחקנים שמעוניינים להכיר באופן תיאורתי את חוקי המשחק.

Board - פעולות הלוח, ציירו, והזזות, הוצאות כדור, כל הפעולות הקשורות ללוח מתבצעות במחלקה זו, שאחראית לו. מחלקה זו בנוסף לכך מאפשר העתקה לצורך ביצוע פעולות של המחשב על הלוח מועתק וחשיבה לעומק.

Cell - מחלקה המארגנת את פקודות הכדורים, חוקי סימונם, בדיקות הזזה, ניהולי מהלכים אפשריים.

ComputerPlayer - מחלקה האחראית לניהול כל מהלכי השחקן הממוחשב, חישוביו ומציאת המהלך הטוב ביותר.

GameFrame - הפריים המרכזי של המשחק, בו מתקבל המידע על המהלך מהמשתמש, חישובי הציור.

HumanPlayer - מחלקת השחקן שיורשת את התכונות מהמחלקה האבסטרקטית שחקן.

Move - מחלקה שמנהלת את המהלכים באופן חכם ושמפשרת את המהלכים בלוח הווירטואלי.

Player - מחלקה אבסטרקטית שחקן, שמכילה את ההרשאות והתכונות המשותפות לשחקנים.

Spot - מחלקת כתם שאחראית לחישוב כוח הכתמים, אחד הגורמים החשובים בחשיבת השחקן הממוחשב.

שימוש במערך-רשימות ArrayList

בפרוייקט שלי יש שימוש רב במערך רשימות. מבנה נתונים זה הוא גמיש ונוח, ומאפשר שינוי ועדכון פשוט של המערך. השימוש הרב שלי בהם נוח מהנוחות של בשימוש במבנה נתונים שמתעדכן בפשוט בעת כל הוספה או הסרה, השגת ערכים במיקומים מסויימים, ואלו מצרכים חשובים בפרוייקט שלי.

השימוש בה לידי ביטוי בכדורים שסומנו ונדחפים, בכדורי אוייב (אם קיימים) שעומדים מול השחקן, בעת ביצוע פעולות של כתמים בחשיבת המחשב, בעת מציעת מהלכים, פעולות שמצריכות שינויים מתמידים כשעובדים ללא ידיעה בכמות המדוברת.

אלגוריתמים תפקוד הלוח והכלים

בדיקת סימון חוקי של כדורים:

אחרי שקבעתי את שיטת הסימון שפורטה לעיל, הבנתי שקיים צורך בדיקת סימון חוקי למען להבטיח שמירה על חוקי המסשחק. אז כמו שנאמר, כל לחיצה על כדור של שחקן שכרגע תורו, משנה את סטטוס הכדור מצבעו הרגיל למסומן בעזרת אלגוריתם חכם שבודק את סוג הכדור ואם סימון זה מתאפשר לפי חוקי המשחק. באלגוריתם מזהה את כל הכדורים כדי לבדוק רציפות שורתית בינהם + סוג של שני כדורים, ושל שלוש כדורים. צבעם של הכדורים משתנה כל עוד לא סומנו יותר מ-3 כדורים והחוקים מתבצעים. האלגוריתם מתייחס לכמות הכדורים בעת קביעת הבדיקה הנדרשת. קודם כל הוא מזהה כדורים צמודים בעזרת ריצה על מטריצת כיוונים, ובמידע וסומנו 3 כדורים, בעזרת חישובים. תחילה, הבחנתי שיש צורך להבדיק בין סימון לארוך, לבין סימון באלכסון. כיוון שבדיקת סימון לאורך הינה חישוב תוצאה מוחלטת בין הכדור הראשון לכדור השני, והבחנה שאין בינם הבדל העולה על 2 ביטיים, ואילו בבדיקה של סימון אלכסוני, יש בנוסף לקחת בחשבון שאסור שכפול של עמודה **או** תור.

להלן פירוט קצת של הבדיקה:

1. אם יש התאמה בין השורה הראשונה לשניה

// בדיקה ראשונית שמזהה אם סוג הבדיקה שתתבצע.

2. אם יש התאמה בין השורה הנוכחית לשורה השנייה

// גם הכדור שכרגע סומן הוא כדור בשורה שאנחנו מסמנים.

3.1 אם הערך המוחלט מכל נקודה לרוחב הוא לכל היותר 2. – מוחלט כי יכול להיות שלילי

// בדיקה אם הכדור הנוכחי שסומן הוא בעצם לא עולה על מרחק של 2 באף צד.

3.2

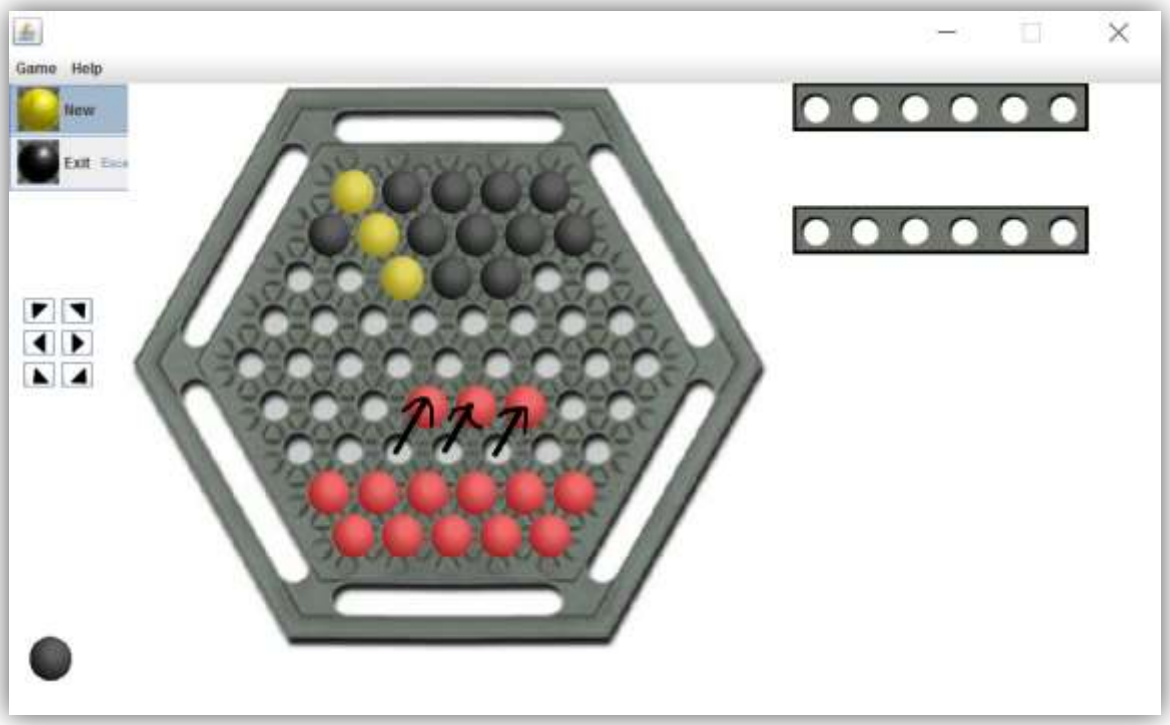
// כעת אנחנו מסתכלים על סימון דיאגנלי(אלכסוני).

4. בדיקה אם המרחק נשמר, אך גם אין שכפול של אף נקודה, כלומר שהנקודה ממשחכה.

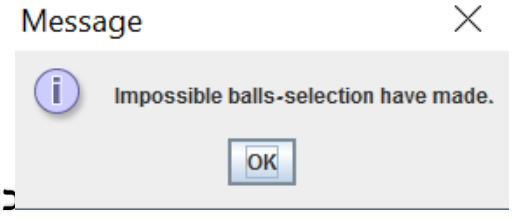
בנוסף לבדיקה אם הכדור הנוכחי שסומן הוא בעצם לא עולה על מרחק של 2 באף צד. //

בדיקת אי-שכפול כמו שצויין. //

לוח אחרי שיחוק חוקי של אדם וסימון חוקי של שחור:



במידע ולא יתבצע סימון נכון, המערכת תבטל סימון מבחינה תכנותית, ותתריאה על



כך:

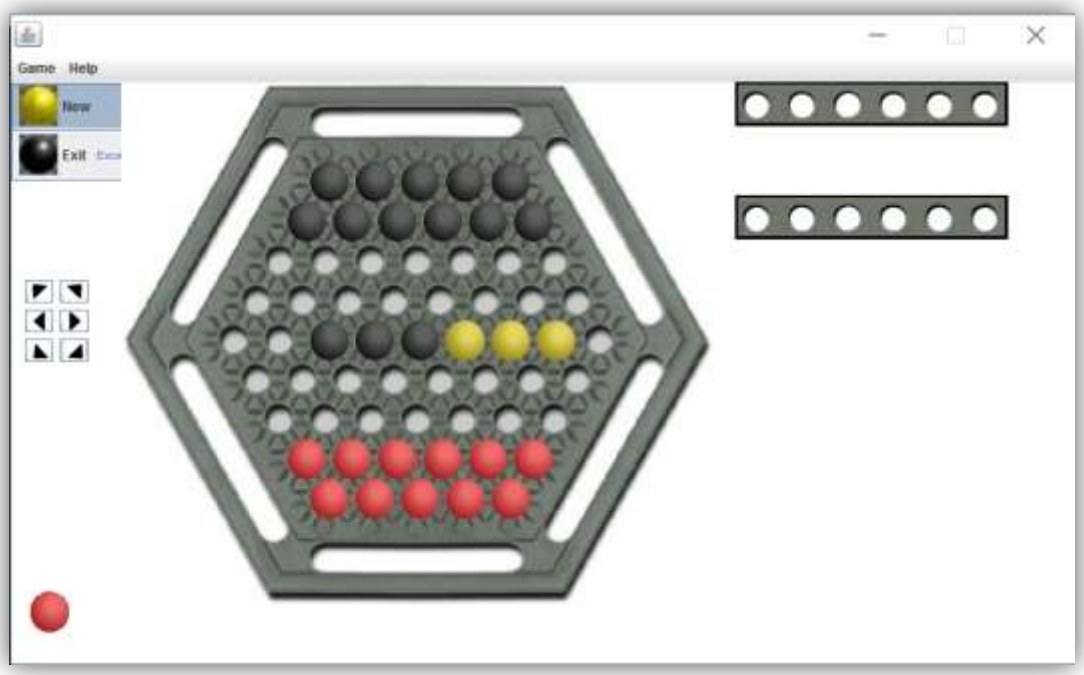
ביצוע לוחמת הזזה:

```
/* ArrayLists */
public ArrayList<Cell> SelectedBalls;
public ArrayList<Cell> Enemy;
```

מתייחסים לשתי צדדים, שתי מערכי-רשימות:

המערכת תשתמש ברשימת מערך אויב בעת התקלות עם אויב, זאת כדי להבטיח שבמידע ואין ביכולת הכדורים שסומנו להזיז את האויב, דבר זה לא יקרה. עם זאת, שבמידע ודחיפה מתאפשרת, אז גם כדורי האויב יזוזו.

להלן המחשה ויזואלית של מה קורה בקוד:



המחשב מוסיף למערך-רשימות האויב את הכדורים מולו, ועד ל3 כדורים (הקרובים ביותר שאפשריים להזזה, או ליתר דיוק הבולמים את ההתקפה).

<u>להלן הקוד לחישובים אלו:</u>

```
public void doStep(int dir) {
    if (SelectedBalls.isEmpty()) { // Function won't move if no balls selected to move.
        javax.swing.JOptionPane.showMessageDialog(Parent, "Bugs, you haven't selected any ball. OK forgot to re-select, select some
        return;
    }
    Object[] ArC = SelectedBalls.toArray(); // ArC - Copied Array which will be used for checks.
    int row = ((Cell) ArC[0]).m_Row;
    int col = ((Cell) ArC[0]).m_Col;
    while (ArCells[row][col] != null && ArCells[row][col].m_Type == ((Cell) ArC[0]).m_Type) { // Skip same type
        row += Direction[Dir][1];
        col += Direction[Dir][0];
    }
    int EnemyType = ((Cell) ArC[0]).m_Type == Cell.SELECTEDBLACK ? Cell.RED : Cell.BLACK; // // Declaration of integer to later chec

    // Check is enemy - if YES -> Building ArrayList to know the enemy's size.
    while (ArCells[row][col] != null && ArCells[row][col].m_Type == EnemyType && Enemy.size()<3) {
        Enemy.add(ArCells[row][col]);
        row += Direction[Dir][1];
        col += Direction[Dir][0];
    }

    /* Check if ball-push can be done. */
    if (ArCells[row][col] != null // The done move isn't going outside.
            && ArCells[row][col].m_Type == Cell.EMPTY // Moving it an empty cell.
            && (SelectedBalls.size() > Enemy.size() // Move done is possible and Enemy's size isn't bigger.
            && Enemy.size() > 0 || Enemy.isEmpty())) { // The enemy is being eaten.

        MoveCells(Dir, null);
        ChangeTurn();
```

<u>פונקציה לשליחה להזה לאחר שאושרה הדחיפה:</u>

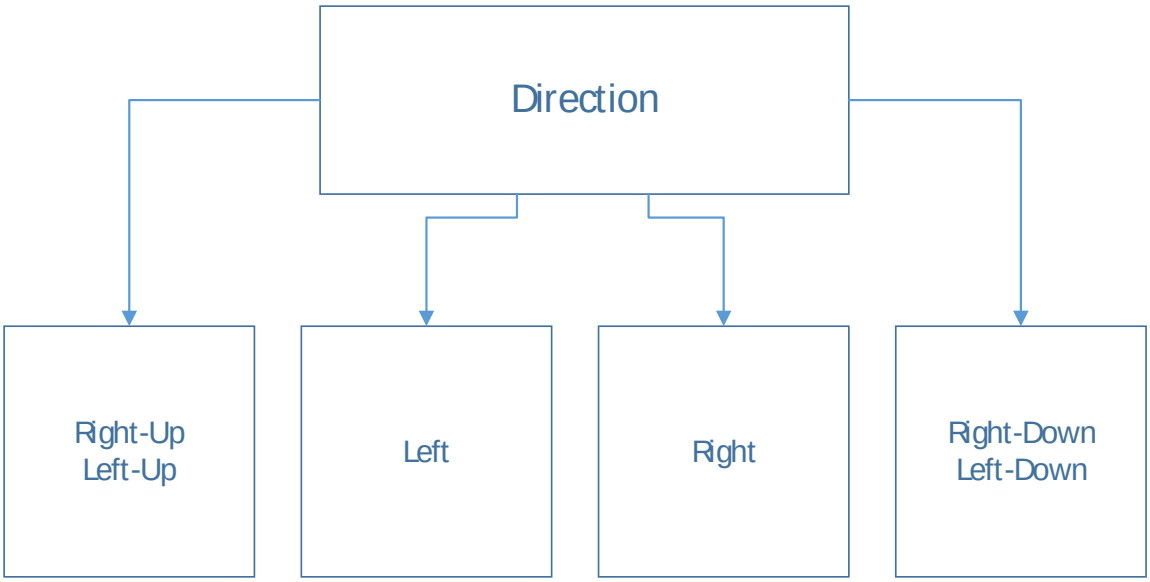
לאחר שנדבקה דחיפה חוקית, יש פונקציה שאליה נשלחות ההודעות בקשר לכיוון לפי מטריצת הכיוונים, ומהלך מחשב אם בוצע ולא ריק:

```
/* Function to set Player and Enemy to sort, to know which set should be set as null, and moves the balls in the given direction.
* Sending messages to make move for AI
    Input: Integer represents the direction, and Comp move.
    Processing: Function re-setting the Selected and Enemy balls, and moving them.*/
public void MoveCells(int Dir, Move Comp) {

    /* Sorting both Selected and Enemy ArrayLists to know which cell to set null */
    Sort(SelectedBalls, Dir);
    Sort(Enemy, Dir);

    /* Moving Enemy's balls, then Player's */
    for (int i = 0; i < Enemy.size(); i++) {
        MoveCell(Enemy.get(i), Dir, Comp);
    }
    for (int i = 0; i < SelectedBalls.size(); i++) {
        MoveCell(SelectedBalls.get(i), Dir, Comp);
    }

    SelectedBalls.clear();
    Enemy.clear();

    Win(); // Check if win.

    if (Comp == null && m_Comp != null) {
        m_Comp.AIDoStep();
    }
}
```

מיון כדורים:

<u>בעיה אלגוריתמית:</u> אחת הווידואיים שתחילה הזנחתי, היה דחיפה תקנית עם תייחסות למצב המערך רשימה המעודכן, והיכן קיים צורך להגדיר את התאים לריק. כיוון שבפיתוח הראשוני שלי הייחסתי בעיקרון לאלכסונים בחישוביי, לא הכרתי בכך שהזזות של כדורים רוחבים מצריכים התייחסות שונה. לכן, כתבתי אלגוריתם שתפקידו להבטיח לי סימון נכון של תאים רקים.

האלגוריתם מיון הוא מאד אישי ומתייחס לסיטואציות השונות שונה, על בסיס מערך הכיוונים:



מוטיבציה של אלגוריתם זה הוא להעתיק את המערך ככה שהאלגוריתם ההזה ידע לפרש אותו בצורה שמחשיבה את ההזה, וכך תוכל לבסוף למנה בתוך המערך של הלוח את התא הריק במקום הנכון. כלומר, הלוח נמחק ומתעדכן לאחרי המיון.

להלן ההתייחסות למיון בקוד:

```java
/* Function sorting the moving balls to know which cells should be set to EMPTY
    Input: ArrayList needed to sort, direction integer.
    Processing: Function using the direction to recognise which treat should the ArrayList have,
depending on it function does the changes.
 */
private void Sort(ArrayList List, int Dir) {
    Object[] Ob = List.toArray(); // Copying the ArrayList given.

    if (Dir < 2) { // Working with RIGHT-UP and LEFT-UP directions.
        for (int i = 0; i < List.size() - 1; i++) {
            for (int j = 0; j < List.size() - 1; j++) {
                if (((Cell) Ob[j]).m_Row > ((Cell) Ob[j + 1]).m_Row) {
                    Object T = Ob[j];
                    Ob[j] = Ob[j + 1];
                    Ob[j + 1] = T;
                }
            }
        }
    }

    if (Dir == 3 || Dir == 4) { // Working with RIGHT-DOWN and LEFT-DOWN directions.
        for (int i = 0; i < List.size() - 1; i++) {
            for (int j = 0; j < List.size() - 1; j++) {
                if (((Cell) Ob[j]).m_Row < ((Cell) Ob[j + 1]).m_Row) {
                    Object T = Ob[j];
                    Ob[j] = Ob[j + 1];
                    Ob[j + 1] = T;
                }
            }
        }
```

```java
    if (Dir == 2) { // Working with RIGHT direction.
        for (int i = 0; i < List.size() - 1; i++) {
            for (int j = 0; j < List.size() - 1; j++) {
                if (((Cell) Ob[j]).m_Col < ((Cell) Ob[j + 1]).m_Col) {
                    Object T = Ob[j];
                    Ob[j] = Ob[j + 1];
                    Ob[j + 1] = T;
                }
            }
        }
    }

    if (Dir == 5) { // Working with LEFT direction.
        for (int i = 0; i < List.size() - 1; i++) {
            for (int j = 0; j < List.size() - 1; j++) {
                if (((Cell) Ob[j]).m_Col > ((Cell) Ob[j + 1]).m_Col) {
                    Object T = Ob[j];
                    Ob[j] = Ob[j + 1];
                    Ob[j + 1] = T;
                }
            }
        }
    }
    List.clear();
    List.addAll(Arrays.asList(Ob));
}
```

הזזת כלים:

פונקציית הזזת הכלים היא פונקציה שתפקידה להזיז את הכדור לאחר שהכדורים מויינו ולבסוף להחליף את המקום המקום הקודם בריק. בנוסף לכך, פונקציה זו בודקת אם כדור נדחף אל מחוץ ללוח, על מנת להוציא מהלוח

להלן הקוד:

```
/* The actual function that runs the moves and moves only selected balls + Removes if balls pushed outside the board.
+  The moves cells are actually sorted to the function knows which cells to update.
    Input: Cell, direction, and Computer move.
    Processing: Function checking if ball pushed out, if yes it removes it, sending message to CPU if played,
Updating the pieces and eventually setting the place the ball moves from to EMPTY.
 */
private void MoveCell(Cell C, int Dir, Move Comp) {
    int row = C.m_Row;
    int col = C.m_Col;
    switch (C.m_Type) {
        case Cell.RED:
        case Cell.SELECTEDRED:
            if (ArCells[row + Direction[Dir][1]][col + Direction[Dir][0]] == null) {
                if (Comp == null || !Comp.virtual) {
                    PutOut(OutRed, Cell.RED, GameFrame.m_Start.y);
                    m_Human.pieces.remove(C);
                } else {
                    Comp.pushOut = true;
                }
            } else {
                ArCells[row + Direction[Dir][1]][col + Direction[Dir][0]].SetCell(Cell.RED);
                m_Human.upDateList(ArCells[row + Direction[Dir][1]][col + Direction[Dir][0]], C);
            }
```

```
        case Cell.BLACK:
        case Cell.SELECTEDBLACK:
            if (ArCells[row + Direction[Dir][1]][col + Direction[Dir][0]] == null) {
                if (Comp == null) {
                    PutOut(OutBlack, Cell.BLACK, GameFrame.m_Start.y + 100);
                    if (Comp != null) {
                        m_Comp.pieces.remove(C);
                    }
                } else {
                    Comp.pushOut = true;
                }
            } else {
                ArCells[row + Direction[Dir][1]][col + Direction[Dir][0]].SetCell(Cell.BLACK);
                if (m_Comp != null) {
                    m_Comp.upDateList(ArCells[row + Direction[Dir][1]][col + Direction[Dir][0]], C);
                }
            }
            break;
    }
    ArCells[row][col].SetCell(Cell.EMPTY);
}
```

בדיקת הוצאת כדור וניצחון:

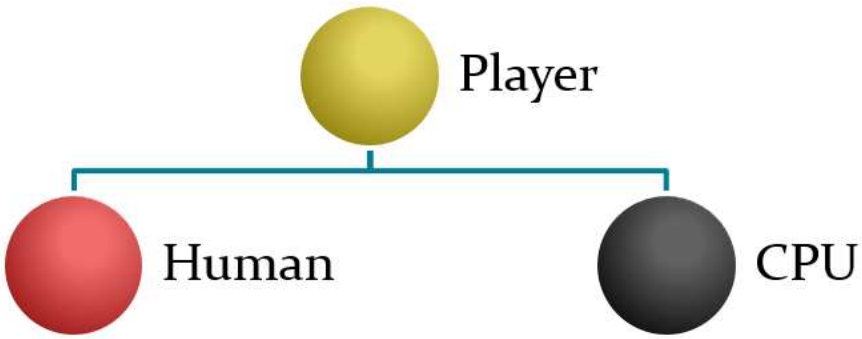
כמו שצויין לעיל, המשחק נגמר בניצחון כששחקן מוציא כ-6 כדורים מכדורי האויב, עם זאת ישנה גם כן פונקציה שאחראית לניהול הציור של הכלים בתבניות הכדורים שהוצאו. להלן פונקציות אלו:

```
/* Function manages the pushed outside balls.
    Input: ArrayList we're drawning, type drawn and y location(There's a difference of 100 between the two bars)
    Processing: Function adds to the drawning the cell and adds ArrayList
 */
private void PutOut(ArrayList<Cell> Out, int T, int y) {
    Cell Temp = new Cell(Parent, 1, 2, T);
    Temp.DrawImage(GameFrame.m_Start.x + Parent.wBoard + 20 + Out.size() * (Cell.CellSizeW + 6), y);
    Out.add(Temp);
}

/* Function checks if any side won the game by pushed out sizes, to annouch so and start a new game */
private void Win() {
    if (OutRed.size() == 6) {
        javax.swing.JOptionPane.showMessageDialog(Parent, "Black player has won the game!");
        Parent.NewGame(Num);
    }
    if (OutBlack.size() == 6) {
        javax.swing.JOptionPane.showMessageDialog(Parent, "Red player has won the game!");
        Parent.NewGame(Num);
    }
}
```

בינה מלאכותית

הבחנה: הורשה של אנוש ומחשב משחקן:

לאחר מכן, כמו שצויין מקודם, יש להתייחס לשחקנים אנושים ולשחקן מחשב כאחד, ולכן נכתבה מחלקה אבסטרקטית בשם שחקן שממנה יורשים שחקן אנושי ומחשב.



המחלקה האבסטרקטית שחקן מכילה את הפריים, מערך-רשימות של כדורים שלרשותו, ומערך רשימות של אפשרויות מהלכים. בנוסף לכל היא מכילה את הפונקציות שדואגות ליצירת גושים, כי לפי מוטיבציית המשחק, מצבו של שחקן ימדד בריכוז כדוריו יחד, והמצאותם במרכז. חישובים אלו חיוניים לחשיבה מחושבת ושיקוליו המתמטיים של מחשב כדי שיוכל לבצע את המהלך הטוב ביותר.

מחלקה אבסטרקטית 'שחקן':

תכונות מחלקה: *פריים, *מערך-רשימות של כלים, *מערך-רשימות של כתמים.

פעולות אבסטרקטיות: פעולה לביצוע מהלך.

פעולות:

יצירת כתמים – אחד הגורמים הקובעים את מעמדו של שחקן הוא שמירתו על מספר כתמים נמוך, ולכן הפעולה יוצרת כתמים לפי כמותם ומאתחלת אותם למערך-רשימות כתמים.

פעולת עזר - לאתחול הכלים ללא נבדקו למען בדיקה.

פעולה ליצירת כתם – יוצרת כתמים באופן רקורסיבי

מחלקת 'שחקן מחשב':

מחלקת המחשב אחראית לצורת מוח המחשב. כלומר, היא מבצעת פעולות בעזרת חישובים ושיקולים שהוכנסו בה.

תכונות מחלקה: *מערך-רישומת של מהלכים אפשריים.

פעולות שמתבצעות על ידי מחשב, נעשת במחלקת בינה המלאכותית אשר יורשת מThread.

פעולה אשר (לעבור על זה)

ושם, מופעלת פעולת הDoStep.

פעולת זו מופעת לבמלקת הבינה המלכותית כדי למנוע מהפעולות להתבצע יחד:

להלן זימון כל המהלכים הקיימים בעזרת קריאה לטוען-הצעדים והגדרת המהלך הטוב שבסופו יבוצע אותו מהלך לאחר מציאתו:

```
public void DoStep() {
    FillMoves(); // Setting up the all possible moves.
    Move BestMove = null; // Best move we're looking for.
    double MaxGrade = -30000.0D; // Minimal possible move.
```

פעולת מילוי המהלכים ממלא לתוך מערך-הרשימות את המהלכים הפנויים:

```
private void FillMoves() {
    for (int i = 0; i < pieces.size(); i++) {
        ((Cell) pieces.get(i)).FillMovesAvaliable(Avaliable);
    }
}
```

לאחר מכן, לכל הפעולות נבדקות על מנת למצוא את הרצוייה ביותר ולאתחל אותה:

```
for (int i = 0; i < Avaliable.size(); i++) {

    ((Move) Avaliable.get(i)).DoMove(m_Parent); // Making a move for Humman in temp
    pieces = m_Parent.m_Board.m_Comp.pieces; // Inits comp balls.
    ((Move) Avaliable.get(i)).grade = GetGrade(); // Calculationg of move grade.
    if (((Move) Avaliable.get(i)).pushOut) {
        ((Move) Avaliable.get(i)).grade += 100.0D; // High motivation to push out.
    }
    ((Move) Avaliable.get(i)).UndoMove(m_Parent);
    pieces = m_Parent.m_Board.m_Comp.pieces; // Sets back comp balls.
    if (((Move) Avaliable.get(i)).grade > MaxGrade) {
        MaxGrade = ((Move) Avaliable.get(i)).grade;
        BestMove = (Move) Avaliable.get(i);
    }
}
```

ובסופו של דבר, אם נמצא פעולה זו, היא מתבצעת ומחשב מחליף בין השחקנים:

```
if (BestMove != null) {
    BestMove.virtual = false; // Not a virtual move and actually made one.
    BestMove.DoMove(m_Parent);
}
m_Parent.m_Board.ChangeTurn(); // Change turn.
```

מחלקת 'מהלך':

מטרת החלקה היא לארגן את המהלכים במשחק. בנוסף לכך, בעזרת מחלקה מתאפשר פיתוח ותחזוקה גמישה יותר של המערכת, מצרך חשוב מאד לתוכנות גדולות שממשיכות להתפתח כל הזמן. המחלקה משתמשת בנתונים הנ"ל לארגון מהלכים נכון:

```
public class Move {

    Point p;
    int dir, attack, enemyAttack;
    double grade;
    boolean pushOut, virtual;
    Board TempBoard = new Board();

    /* Move constractor */
    public Move(Point sour, int dir, int attack, int enemyAttack, boolean pushOut) {
        this.p = sour; // Source point.
        this.dir = dir; // Direcion.
        this.attack = attack; // Self-Attack power by size of balls selected.
        this.enemyAttack = enemyAttack; // Number of balls blocking the attack.
        this.pushOut = pushOut; // Boolean to tell if ball being pushed out.
        this.virtual = true; // Boolean if move isn't really being made, but for ..
        // .. the check of AI. Default option is 'true'.
    }
```

(משתנים גלובים + בניי מחלקת המהלך)

לניהול המהלכים, המחלקה כוללת את הפונקציות הבאות:

<u>פונקציה לביצוע מהלכים בלוח המועתק:</u>

```
public void DoMove(GameFrame g) {
    TempBoard.CopyBoard(g, g.m_Board.ArCells); // Copying current board.

    /* Clearing balls */
    g.m_Board.SelectedBalls.clear();
    g.m_Board.Enemy.clear();

    Cell[][] mat = g.m_Board.ArCells; // New Cells array based on pieces.
    int row = p.y, col = p.x; // Having location of clicked point..s
    for (int i = 0; i < attack; i++) {
        g.m_Board.SelectedBalls.add(mat[row][col]);
        row += Board.Direction[dir][1];
        col += Board.Direction[dir][0];
    }
    for (int i = 0; i < enemyAttack; i++) {
        g.m_Board.Enemy.add(mat[row][col]);
        row += Board.Direction[dir][1];
        col += Board.Direction[dir][0];
    }
    g.m_Board.MoveCells(dir, this); // Move being in a copied board.
```

<u>פונקצית ביטול על מנת לחזור לפעולה על הלוח המקורי:</u>

```java
public void UndoMove(GameFrame Parent) {
    Parent.m_Board.CopyBoard(Parent, TempBoard.ArCells);
}
```

יש ציין כי לאחר ביצוע פעולה, הלוח מאותחל עד לביצוע כל הפעולות האפשריות, והפעולה שנמצא להכי טובה תיהיה היחידה שלא תשוחזר. כלומר, הפעולות מתבצעות על הלוח מועתק אומנם המועתק מאותחל למקורי לאחר מציאת פעולה זו.

להלן האלגוריתם להעתקת לוח:

```java
/* Empty Constractor for when Copying board and doing moves on it */
public Board() {
}

/* CopyBoard function will be used to copy the board to the blank board created above. Board is based on currect board.
    Input: Function recieving the GameFrame, and the Array of Cells.
    Processing: Function Creating Human and Player, and for cells it inits back the pieces of each party.
so what's literally being done is a Copied board.
 */
public void CopyBoard(GameFrame Parent, Cell[][] Mat) {
    m_Human = new HumanPlayer(Parent);
    m_Comp = new ComputerPlayer(Parent);
    ArCells = new Cell[ROWS][COLS];
    for (int i = 0; i < ROWS; i++) {
        for (int j = 0; j < COLS; j++) {
            if (Mat[i][j] != null) {
                ArCells[i][j] = new Cell(Parent, Mat[i][j].m_Row, Mat[i][j].m_Col, Mat[i][j].m_Type);
                if (Mat[i][j].m_Type == Cell.BLACK) {
                    m_Comp.pieces.add(ArCells[i][j]);
                }
                if (Mat[i][j].m_Type == Cell.RED) {
                    m_Human.pieces.add(ArCells[i][j]);
                }
            } else {
                ArCells[i][j] = null;
            }
        }
    }
}
```

מחלקת 'כתם':

מהו כתם? בכתם הכוונה לכדורים צמודים יחד. פעולה זאת מאותחלת במחלקה האבסטרקטית 'שחקן' כיוון שהיא משותפת לשני הצדדים. להלן הפעולה הרקורסיבית ליצירת כתם:

להלן הקריאה לפעולה:

```
public void createSpots() {
    InitCells(); // Sets for all pieces Visited status to false.
    spotList.clear(); // Clearning previous created spots.
    Spot s = new Spot();
    spotList.add(s);
    makeSpot((Cell) pieces.get(0), ((Cell) pieces.get(0)).m_Type); // Function recursively calculates the spot.
    ((Spot) spotList.get(0)).setGrade(); // Setting grade for the currect spot.
    for (int i = 1; i < pieces.size(); i++) {
        if (((Cell) pieces.get(i)).bolVisited == false) { /* Creating spot for unvisited spots */
            spotList.add(new Spot());
            makeSpot((Cell) pieces.get(i), ((Cell) pieces.get(i)).m_Type);
            ((Spot) spotList.get(spotList.size() - 1)).setGrade(); // Setting grade.
        }
    }
}
```

להלן יצירת הכתמים הרקורסיבית:

```
private void makeSpot(Cell cell, int type) {
    if (cell == null) {
        return;
    }
    Cell[][] m = m_Parent.m_Board.ArCells; // New Cells array, based on real one.
    int row = cell.m_Row;
    int col = cell.m_Col;

    if ((cell.m_Type != type) || cell.bolVisited) { // Spot already made.
        return;
    }
    ((Spot) spotList.get(spotList.size() - 1)).listSpot.add(cell); // Else - Make the spot
    m[row][col].bolVisited = true;

    /* Making spot in all directions recursively. */
    makeSpot(m[Board.Direction[0][1] + row][Board.Direction[0][1] + col], type);
    makeSpot(m[Board.Direction[1][1] + row][Board.Direction[0][1] + col], type);
    makeSpot(m[Board.Direction[2][1] + row][Board.Direction[0][1] + col], type);
    makeSpot(m[Board.Direction[3][1] + row][Board.Direction[0][1] + col], type);
    makeSpot(m[Board.Direction[4][1] + row][Board.Direction[0][1] + col], type);
    makeSpot(m[Board.Direction[5][1] + row][Board.Direction[0][1] + col], type);
}
```

אחת המוטיבציות של המחשב הוא לשמור על כמות כתמים נמוכה, כיוון שכמות כתמים נמוכה מבטיחה השארות כמות כדורים יחד, ועם זאת מבטיחה שהכוח מרוכז וככה בעצם חזק יותר. יש להבהיר.

שני <u>הערכים הגלובים</u> של מחלקת כתמים היא מערך-רשימה שמצייג את כל הכתמים ביחד בעזרת מציאת כתמים רקורסיבית כמו שצויין לעיל.

<u>חישוב ניקוד הכתם:</u>

```
double dDisCol = distanceColCal(((Cell) listSpot.get(0)).m_Col, dCol);
double dMaxDis = Math.sqrt(Math.pow(dCol, 2) + Math.pow((((Cell) listSpot.get(0)).m_Row - dRow), 2)); //
for (int i = 1; i < listSpot.size(); i++) {
    dDisCol = distanceColCal(((Cell) listSpot.get(i)).m_Col, dCol);
    double dis = Math.sqrt(Math.pow(dDisCol, 2) + Math.pow((((Cell) listSpot.get(i)).m_Row - dRow), 2));
    if (dis > dMaxDis) {
        dMaxDis = dis;
    }
}
if (dMaxDis == 0.0) {
    dMaxDis = 1.0; // Setting minimal grade.
}
SpotGrade = listSpot.size() / dMaxDis;
dDisCol = distanceColCal(10.0, dCol);
```

ניתן להבחין גם כן בחישוב הכתם ממרכז הלוח, כיוון שכפי שצויין, המוטיבציה של המשחק היא שהכתמים התרכזו במרכז:

```
double dCenterDis = Math.sqrt(Math.pow(dDisCol, 2) + Math.pow((5.0D - dRow), 2));
if (dCenterDis == 0.0) {
    dCenterDis = 1.0;
}
SpotGrade += 2 * listSpot.size() / dCenterDis;
```

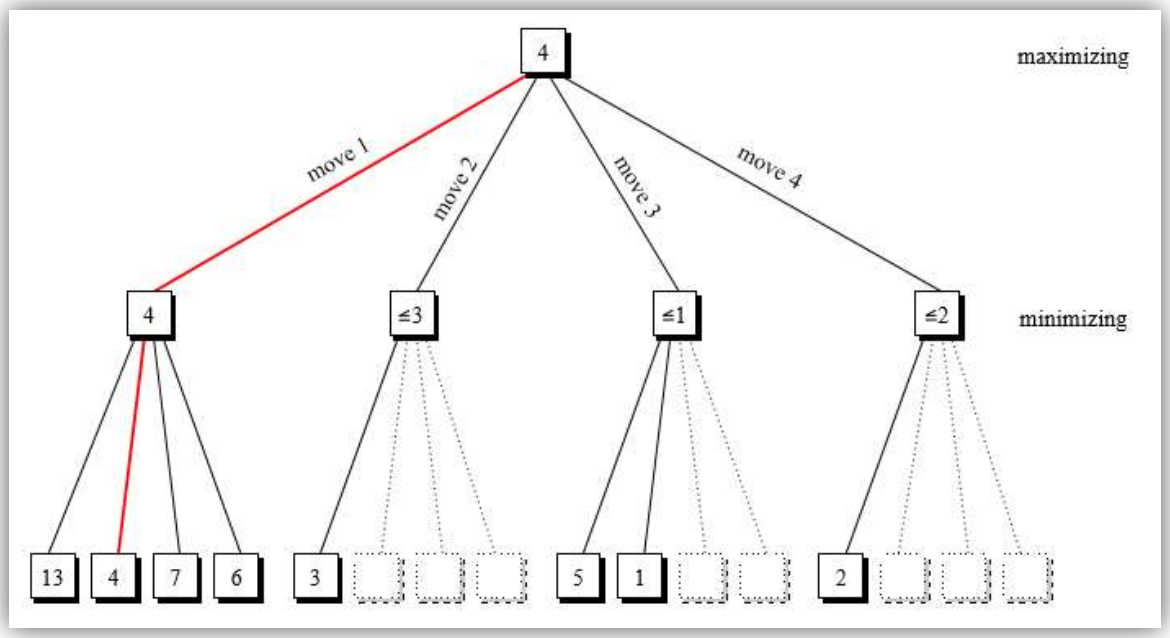
<u>המוטיבציה לשליטה במרכז:</u> הסיבה לרצון בהתרכזות במרכז היא כיוון שאם הכדורים אסופים היטב במרכז הלוח, ניתן להבטיח שכמות כוח גדולה מרוכזת יחד, והכוח רחוק מהשוליים מה שמבטיח את בטיחותו.

חשיבת מינ-מקס:

החישוב מינימקס מתבצע לאחר חשיבה וירטואלית של המחשב למהלכים על מנת להגיע למהלך הטוב ביותר על פי קביעת ציון לכל מהלך ומוטיבציה להגיע למצב האידיאלי למחשב. יש לקחת בחשבון שהגעה למצב זה אינה וודאית, ועדיין תלויה במהלכים של השחקן האנושי. להלן החישוב בתוך האלגוריתם החישוב בפרוייקט:

```
private double GetGrade() {
    createSpots(); // Abaract function that creates the spots of CPU.
    double grade = 0.0; // Decleration double for CPU grade.
    for (int i = 0; i < spotList.size(); i++) { // Calculate grade for every spot.
        grade += ((Spot) spotList.get(i)).SpotGrade;
    }
    grade /= spotList.size(); // Calculation recognizing the less spots, ..
                              // .. higher grade.
    /* Creating the spots for Human and calculating them. */
    m_Parent.m_Board.m_Human.createSpots();
    double HumGrade = 0.0; // Decleration double for Human grade.
    for (int i = 0; i < m_Parent.m_Board.m_Human.spotList.size(); i++) {
        HumGrade += ((Spot) m_Parent.m_Board.m_Human.spotList.get(i)).SpotGrade;
    }
    return ((m_Parent.m_Board.m_Human.spotList.size() - spotList.size()) * 4 + grade - HumGrade);
}
```

<u>להלן המחשה גרפית של עץ מינמקס:</u>



(Graz University of Technology התמונה נלקחה ממאמר של)

השחקן מבצע חישובים אלו בכל פעם שהוא משחק, מה שמקנה לו אפשרות לפרש את המשחק היטב וכמה צעדים קדימה. חישובים אלו בפרוייקט מכונים כ-וויזואלים. כיוון שאין הם אכן מתבצעים אלא רק במוחו של הבינה המלכותית, על מנת לחזות את המהלך הטוב ביותר.

ניספח – קוד התוכנית:

Board:

```
package abalonegame;

import java.awt.*;

import java.util.ArrayList;

import java.util.Arrays;


public class Board {


   GameFrame Parent;

   public static Point m_Start;


   /* Sizes of Rows and Cols in the board. */

   public static final byte ROWS = 11;

   public static final byte COLS = 21;


   /* Images */

   public static Image RedBallImage, BlackBallImage, SelectedBallImage;
```

```
/* Setting global statments */
public static final byte EMPTY = 1;
public static final byte BLACK = 2;
public static final byte RED = 3;
public static final byte SELECTED = 4;

/* ArrayLists */
public ArrayList<Cell> SelectedBalls;
public ArrayList<Cell> Enemy;
ArrayList<Cell> OutRed, OutBlack;

Cell[][] ArCells; // The Boards by cells.

public static boolean Turn = false; // To keep the following after every turn.

/* Players - Human and CPU */
HumanPlayer m_Human;
ComputerPlayer m_Comp;

/* InitBoard for the board the painting */
public static byte[][] InitBoard = {
   {0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0},
   {0, 0, 0, 0, 0, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 0, 0, 0, 0, 0},
   {0, 0, 0, 0, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 0, 0, 0, 0},
   {0, 0, 0, 0, 1, 0, 1, 0, 2, 0, 2, 0, 2, 0, 1, 0, 1, 0, 0, 0, 0},
```

```
    {0, 0, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 0, 0},
    {0, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 0},
    {0, 0, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 0, 0},
    {0, 0, 0, 0, 1, 0, 1, 0, 3, 0, 3, 0, 3, 0, 1, 0, 1, 0, 0, 0, 0},
    {0, 0, 0, 0, 0, 3, 0, 3, 0, 3, 0, 3, 0, 3, 0, 3, 0, 0, 0, 0, 0},
    {0, 0, 0, 0, 0, 0, 3, 0, 3, 0, 3, 0, 3, 0, 3, 0, 0, 0, 0, 0, 0},
    {0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0}
};
private int Num;
/* Possible directions array to apply ball moves based on the selected direction */
static int[][] Direction
        = {// dx dy
            {-1, -1}, // LEFTUP
            {1, -1}, // RIGHTUP
            {2, 0}, // RIGHT
            {1, 1}, // RIGHTDOWN
            {-1, 1}, // LEFTDOWN
            {-2, 0} // LEFT
        };

/* Creates a new instance of Board */
public Board(GameFrame Parent, Point Start, int Num) {
    /* Setting the images to to know to each image we relate */
    RedBallImage = Parent.getToolkit().createImage("Red.png");
    BlackBallImage = Parent.getToolkit().createImage("Black.png");
    SelectedBallImage = Parent.getToolkit().createImage("Selected.png");
```

```
m_Human = new HumanPlayer(Parent);

if (Num == 1) { // Creating a computer player in case user selected vs Computer
    this.m_Comp = new ComputerPlayer(Parent);
}

/* Setting values recieved from frame. */
this.Parent = Parent;
m_Start = Start;
this.Num = Num;

SelectedBalls = new ArrayList<Cell>();  // Own Selected ArrayList
Enemy = new ArrayList<Cell>();          // Enemy balls  ArrayList
ArCells = new Cell[ROWS][COLS];         // Cells array
OutRed = new ArrayList<Cell>();         // Lost RED   balls
OutBlack = new ArrayList<Cell>();       // Lost BLACK balls

/* Painting the cells based on our InitBoard decerated before. */
for (int i = 0; i < ROWS; i++) {
    for (int j = 0; j < COLS; j++) {

        if (InitBoard[i][j] >= 1) {
            ArCells[i][j] = new Cell(Parent, i, j, InitBoard[i][j]);
            ArCells[i][j].DrawImage(Board.m_Start.x + 87, Board.m_Start.y + 70);
            if (InitBoard[i][j] == 2) {
```

```
                if (m_Comp != null) { // Adding balls to Computer if user decided to play him.
                    this.m_Comp.pieces.add(ArCells[i][j]);
                }
            }
            if (InitBoard[i][j] == 3) { // Adding balls to Human.
                this.m_Human.pieces.add(ArCells[i][j]);
            }
        } else {
            ArCells[i][j] = null; // Setting empty spots.
        }


    }
  }
}


/* Empty Constractor for when Copying board and doing moves on it */
public Board() {
}
/* CopyBoard function will be used to copy the board to the blank board created above.
Board is based on currect board.
    Input; Function recieving the GameFrame, and the Array of Cells.
    Processing; Function Creating Human and Player, and for cells it inits back the pieces of
each party,
so what's literally being done is a Copied board.
 */
public void CopyBoard(GameFrame Parent, Cell[][] Mat) {
    m_Human = new HumanPlayer(Parent);
```

```
        m_Comp = new ComputerPlayer(Parent);
        ArCells = new Cell[ROWS][COLS];
        for (int i = 0; i < ROWS; i++) {
            for (int j = 0; j < COLS; j++) {
                if (Mat[i][j] != null) {
                    ArCells[i][j] = new Cell(Parent, Mat[i][j].m_Row, Mat[i][j].m_Col,
Mat[i][j].m_Type);
                    if (Mat[i][j].m_Type == Cell.BLACK) {
                        m_Comp.pieces.add(ArCells[i][j]);
                    }
                    if (Mat[i][j].m_Type == Cell.RED) {
                        m_Human.pieces.add(ArCells[i][j]);
                    }
                } else {
                    ArCells[i][j] = null;
                }
            }
        }
    }

    /* Function manages the graphic drawings.
        Input; Function recieves all graphics. to draw, by size calculation.
        Processing; Function running through all row and cols and draws according to the given
type, what needed.
    Firstly, Function draws all cells, after that function draws the balls.
    */
    public void Draw(Graphics Gr) {
```

```
        /* Balls drawning */
        for (int i = 0; i < ROWS; i++) {
            for (int j = 0; j < COLS; j++) {
                if (ArCells[i][j] != null && ArCells[i][j].m_Type > 1) {
                    ArCells[i][j].DrawImage(Board.m_Start.x + 87, Board.m_Start.y + 70);
                }
            }
        }
        /* Balls poped out drawnings. */
        for (int i = 0; i < OutRed.size(); i++) {
            OutRed.get(i).DrawImage(GameFrame.m_Start.x + Parent.wBoard + 25 + i *
(Cell.CellSizeW + 4), 60);
        }
        for (int i = 0; i < OutBlack.size(); i++) {
            OutBlack.get(i).DrawImage(GameFrame.m_Start.x + Parent.wBoard + 25 + i *
(Cell.CellSizeW + 4), 160);
        }
    }

    /* Function recieving a click location, checking if legal and if so, sends a message to Cell to
click. */
    public void Click(Point point) {
        int row = (point.y - m_Start.y - 70) / 35 + 1;
        int col = (point.x - m_Start.x - 87) / 20 + 2;
        if (IsLegal(row, col) && ArCells[row][col] != null) {
            ArCells[row][col].ClickCell(); // ClickCell in Cell class, checking if the click is legal, and
selecting based on situation.
        }
```

```
}

/* Function checking if the made click is in board's size */
private boolean IsLegal(int row, int col) {
    return row >= 0 && row < ROWS && col >= 0 && col < COLS;
}

/* Changing Turn function, also changing who's turn box in GUI */
public void ChangeTurn() {
    Turn = !Turn;
    Parent.Turn = Turn ? Parent.getToolkit().createImage("Black.png")
            : Parent.getToolkit().createImage("Red.png");
    Parent.repaint();
}

/* Function gets from main a direction to move
    Input; Integer Dir which represents the move being made, later will be translate in the Direction matrix.
    Processing; Function does not go for non-selected clicks. Then function copies the array for the type tests,
  finding the enemy if any to count his power. If move is legal, function sends message to MoveCells to start
  the movment, otherwise it alerts the error via a MessageDialog.
 */
public void DoStep(int Dir) {
    if (SelectedBalls.isEmpty()) { // Function won't move if no balls selected to move.
        javax.swing.JOptionPane.showMessageDialog(Parent, "Woops, you haven't selected any ball, OR forgot to re-select, select some by clicking on them before moving.");
```

```
        return;
    }
    Object[] ArC = SelectedBalls.toArray(); // ArC - Copied Array which will be used for
checks.
    int row = ((Cell) ArC[0]).m_Row;
    int col = ((Cell) ArC[0]).m_Col;
    while (ArCells[row][col] != null && ArCells[row][col].m_Type == ((Cell)
ArC[0]).m_Type) { //  Skip same type
        row += Direction[Dir][1];
        col += Direction[Dir][0];
    }
    int EnemyType = ((Cell) ArC[0]).m_Type == Cell.SELECTEDBLACK ? Cell.RED :
Cell.BLACK; // // Decleration of integer to later check which is enemy.
    //  Check is enemy - IF YES -> Building ArrayList to know the enemy's size.
    while (ArCells[row][col] != null && ArCells[row][col].m_Type == EnemyType &&
Enemy.size()<3) {
        Enemy.add(ArCells[row][col]);
        row += Direction[Dir][1];
        col += Direction[Dir][0];
    }
    /* Check if ball-push can be done. */
    if (ArCells[row][col] != null // The done move isn't going outside.
            && ArCells[row][col].m_Type == Cell.EMPTY // Moving it an empty cell.
            && (SelectedBalls.size() > Enemy.size() // Move done is possible and Enemy's size
isn't bigger.
            && Enemy.size() > 0 || Enemy.isEmpty())) { // The enemy is being moved.
        MoveCells(Dir, null);
        ChangeTurn();
```

```
    } else { // Recognizing another illegal move - pushing ball outside the board.
      if (ArCells[row][col] == null) { // Going for checks if the
        if (Enemy.isEmpty()) { // Confiration the next move is outside the board.
          javax.swing.JOptionPane.showMessageDialog(Parent, "Illegal - Pushing own ball
outside the board is against the game motivation, as you might lose it.");
          javax.swing.JOptionPane.showMessageDialog(Parent, "Remainder: Unselect your
balls to go for a new selection.");
        } else {
          if (SelectedBalls.size() > Enemy.size() && Enemy.size() > 0) {
            MoveCells(Dir, null);
            ChangeTurn();
          }
        }
      } else {
        javax.swing.JOptionPane.showMessageDialog(Parent, "Illegal - Attempt to go on
impossible push. To check the rules press 'H'. ");
        javax.swing.JOptionPane.showMessageDialog(Parent, "Remainder: Unselect your
balls to go for a new selection.");
      }
    }
    SelectedBalls.clear();
    Enemy.clear();
  }

  /* Function to set Player and Enemy to sort, to know which set should be set as null, and
moves the balls in the given direction.
  + Sending messages to make move for AI
    Input; Integer represents the direction, and Comp move.
```

```
    Processing; Function re-setting the Selected and Enemy balls, and moving them.*/
  public void MoveCells(int Dir, Move Comp) {
    /* Sorting both Selected and Enemy ArrayLists to know which cell to set null */
    Sort(SelectedBalls, Dir);
    Sort(Enemy, Dir);

    /* Moving Enemy's balls, then Player's */
    for (int i = 0; i < Enemy.size(); i++) {
      MoveCell(Enemy.get(i), Dir, Comp);
    }
    for (int i = 0; i < SelectedBalls.size(); i++) {
      MoveCell(SelectedBalls.get(i), Dir, Comp);
    }
    SelectedBalls.clear();
    Enemy.clear();

    Win(); // Check if win.
    if (Comp == null && m_Comp != null) {
      m_Comp.AIDoStep();
    }
  }

  /* Function sorting the moving balls to know which cells should be set to EMPTY
    Input; ArrayList needed to sort, direction integer.
    Processing; Function using the direction to recognise which treat should the ArrayList
have,
  depending on it function does the changes.
```

```
*/
private void Sort(ArrayList List, int Dir) {
    Object[] Ob = List.toArray(); // Copying the ArrayList given.
    if (Dir < 2) { // Working with RIGHT-UP and LEFT-UP directions.
        for (int i = 0; i < List.size() - 1; i++) {
            for (int j = 0; j < List.size() - 1; j++) {
                if (((Cell) Ob[j]).m_Row > ((Cell) Ob[j + 1]).m_Row) {
                    Object T = Ob[j];
                    Ob[j] = Ob[j + 1];
                    Ob[j + 1] = T;
                }
            }
        }
    }

    if (Dir == 3 || Dir == 4) { // Working with RIGHT-DOWN and LEFT-DOWN directions.
        for (int i = 0; i < List.size() - 1; i++) {
            for (int j = 0; j < List.size() - 1; j++) {
                if (((Cell) Ob[j]).m_Row < ((Cell) Ob[j + 1]).m_Row) {
                    Object T = Ob[j];
                    Ob[j] = Ob[j + 1];
                    Ob[j + 1] = T;
                }
            }
        }
    }
```

```
    if (Dir == 2) { // Working with RIGHT direction.
        for (int i = 0; i < List.size() - 1; i++) {
            for (int j = 0; j < List.size() - 1; j++) {
                if (((Cell) Ob[j]).m_Col < ((Cell) Ob[j + 1]).m_Col) {
                    Object T = Ob[j];
                    Ob[j] = Ob[j + 1];
                    Ob[j + 1] = T;
                }
            }
        }
    }
    if (Dir == 5) { // Working with LEFT direction.
        for (int i = 0; i < List.size() - 1; i++) {
            for (int j = 0; j < List.size() - 1; j++) {
                if (((Cell) Ob[j]).m_Col > ((Cell) Ob[j + 1]).m_Col) {
                    Object T = Ob[j];
                    Ob[j] = Ob[j + 1];
                    Ob[j + 1] = T;
                }
            }
        }
    }
    List.clear();
    List.addAll(Arrays.asList(Ob));
}
```

```
/* The actual function that runs the moves and moves only selected balls + Removes if
balls pushed outside the board.

+ The moves cells are actually sorted to the function knows which cells to update.

  Input: Cell, direction, and Computer move.

  Processing; Function checking if ball pushed out, if yes it removes it, sending message to
CPU if played,

Updating the pieces and eventually setting the place the ball moves from to EMPTY.

 */

private void MoveCell(Cell C, int Dir, Move Comp) {

  int row = C.m_Row;

  int col = C.m_Col;

  switch (C.m_Type) {

    case Cell.RED:

    case Cell.SELECTEDRED:

      if (ArCells[row + Direction[Dir][1]][col + Direction[Dir][0]] == null) {

        if (Comp == null || !Comp.virtual) {

          PutOut(OutRed, Cell.RED, GameFrame.m_Start.y);

          m_Human.pieces.remove(C);

        } else {

          Comp.pushOut = true;

        }

      } else {

        ArCells[row + Direction[Dir][1]][col + Direction[Dir][0]].SetCell(Cell.RED);

        m_Human.upDateList(ArCells[row + Direction[Dir][1]][col + Direction[Dir][0]],
C);

      }

      break;
```

```
        case Cell.BLACK:
        case Cell.SELECTEDBLACK:
          if (ArCells[row + Direction[Dir][1]][col + Direction[Dir][0]] == null) {
            if (Comp == null) {
              PutOut(OutBlack, Cell.BLACK, GameFrame.m_Start.y + 100);
              if (Comp != null) {
                m_Comp.pieces.remove(C);
              }
            } else {
              Comp.pushOut = true;
            }
          } else {
            ArCells[row + Direction[Dir][1]][col + Direction[Dir][0]].SetCell(Cell.BLACK);
            if (m_Comp != null) {
              m_Comp.upDateList(ArCells[row + Direction[Dir][1]][col + Direction[Dir][0]],
C);
            }
          }
          break;
      }
      ArCells[row][col].SetCell(Cell.EMPTY);
    }


    /* Function manages the pushed outside balls.
      Input; ArrayList we're drawning, type drawn and y location(There's a difference of 100
between the two bars)
      Processing; Function adds to the drawning the cell and adds ArrayList
```

```
    */
    private void PutOut(ArrayList<Cell> Out, int T, int y) {
        Cell Temp = new Cell(Parent, 1, 2, T);
        Temp.DrawImage(GameFrame.m_Start.x + Parent.wBoard + 20 + Out.size() *
(Cell.CellSizeW + 6), y);
        Out.add(Temp);
    }

    /* Function checks if any side won the game by pushed out sizes, to annouch so and start
a new game */
    private void Win() {
        if (OutRed.size() == 6) {
            javax.swing.JOptionPane.showMessageDialog(Parent, "Black player has won the
game!");
            Parent.NewGame(Num);
        }
        if (OutBlack.size() == 6) {
            javax.swing.JOptionPane.showMessageDialog(Parent, "Red player has won the
game!");
            Parent.NewGame(Num);
        }
    }
}
```

Cell:

```
package abalonegame;
import java.awt.Graphics;
import java.awt.Point;
```

```
import java.util.ArrayList;

public class Cell {
    /* Decleration of global and class values */
    public static final int EMPTY = 1;
    public static final int BLACK = 2;
    public static final int RED = 3;
    public static final int SELECTEDBLACK = 4;
    public static final int SELECTEDRED = 5;
    public static int CellSizeW = 34;
    public static int CellSizeH = 36;
    public static int DirectionSize = 6;
    public int m_Row;
    public int m_Col;
    public int m_Type;
    GameFrame Parent;
    boolean bolVisited;

    /*
     * Cell Constractor - Creates a new instance of Cell
     */
    public Cell(GameFrame Parent, int row, int col, int Type) {
        this.Parent = Parent;
        m_Row = row;
        m_Col = col;
        m_Type = Type;
```

```
        bolVisited = false;
    }
    /* Function recieves the type and sends message to DrawImage to draw the imgane in
given directions.
        Input: Type.
        Processing: Setting the type of the cell, and sending location to draw to DrawImage
function.
    */
    public void SetCell(int Type) {
        m_Type = Type;
        DrawImage(Board.m_Start.x + 87, Board.m_Start.y + 70);
    }
    /* Function painting balls
        Input: (x;y) Locations.
        Processing: Depending on the type, Image being drawn.
    */
    public void DrawImage(int x, int y) {
        Graphics Gr = Parent.getGraphics();
        switch (m_Type) {
            case BLACK:
                Gr.drawImage(Board.BlackBallImage, x + (m_Col - 2) * 20, y + (m_Row - 1) * 35,
CellSizeW, CellSizeH, Parent);
                break;
            case RED:
                Gr.drawImage(Board.RedBallImage, x + (m_Col - 2) * 20, y + (m_Row - 1) * 35,
CellSizeW, CellSizeH, Parent);
                break;
            case SELECTEDRED:
```

```
            Gr.drawImage(Board.SelectedBallImage, x + (m_Col - 2) * 20, y + (m_Row - 1) * 35,
CellSizeW, CellSizeH, Parent);

            break;

          case SELECTEDBLACK:

            Gr.drawImage(Board.SelectedBallImage, x + (m_Col - 2) * 20, y + (m_Row - 1) * 35,
CellSizeW, CellSizeH, Parent);

            break;

      }

   }

   /* Function checking size selected not above 3, so if so Function won't let go

      Input: Non.

      Processing: Function checking the size and returning false of size is above 3, what's
against the rules.

    */

   private boolean CheckPossibleSize() {

      return Parent.m_Board.SelectedBalls.size() < 3;

   }

   /* Function anaging the selections bzsed on user click.

      Input; Non*

      Processing; Function checking via math calculations if the selection made is possible,
After that sends.

   the recuired messages to draw the image

    */

   public void ClickCell() {

      int S = Parent.m_Board.SelectedBalls.size() - 1,

            i = -1, // Check for first value as 'i' will be used later.

            R, C, T, R2, C2, T2; // Declecered integers which will be used later in function.

      boolean sec = false,
```

```
            thr = true;
        if (Board.Turn && m_Type == RED || !Board.Turn && m_Type == BLACK) { // Won't go
if no meet with turn and selection.
            return;
        }
        if (S >= 0) {
            R = Parent.m_Board.SelectedBalls.get(0).m_Row;
            C = Parent.m_Board.SelectedBalls.get(0).m_Col;
            T = Parent.m_Board.SelectedBalls.get(0).m_Type;
            if (S == 0) {
                if (m_Type != T) {// Move-check will be based on first ball selected.
                    for (i = 0; i < DirectionSize; i++) // Going though the directions and checking if
move allowed.
                    {
                        if (R + Board.Direction[i][1] == m_Row
                                && C + Board.Direction[i][0] == m_Col) { // Direction right.
                            sec = true;
                        }
                    }
                }
            } else if (S == 1) {
                R2 = Parent.m_Board.SelectedBalls.get(1).m_Row;
                C2 = Parent.m_Board.SelectedBalls.get(1).m_Col;
                T2 = Parent.m_Board.SelectedBalls.get(1).m_Type;
                if (m_Type != T2) { // Selected ball is not already selected.
                    if (R == R2) {
                        if (m_Row == R) {
```

```
                if ((Math.abs(m_Col - C2) > 2 && (Math.abs(m_Col - C) > 2))) {
                    thr = false;
                }
            } else {
                thr = false;
            }
        } else if ((Math.abs(m_Col - C2) > 2 && (Math.abs(m_Col - C) > 2)) || m_Row == R
|| m_Row == R2 || m_Col == C || m_Col == C2) {
            thr = false;
        }
    }
  }
}

if ((!sec && i != -1) || !thr) {
    javax.swing.JOptionPane.showMessageDialog(Parent, "Impossible balls-selection
have made.");
    javax.swing.JOptionPane.showMessageDialog(Parent, "Remainder:  Unselect your
balls to go for a new selection.");
    return;
}

/* Selection */
switch (m_Type) {
    case BLACK:
        if (CheckPossibleSize()) {
            m_Type = SELECTEDBLACK;
```

```
            Parent.m_Board.SelectedBalls.add(this);
        }
        break;
    case RED:
        if (CheckPossibleSize()) {
            m_Type = SELECTEDRED;
            Parent.m_Board.SelectedBalls.add(this);
        }
        break;
    case SELECTEDRED:
        m_Type = RED;
        Parent.m_Board.SelectedBalls.remove(this);
        break;
    case SELECTEDBLACK:
        m_Type = BLACK;
        Parent.m_Board.SelectedBalls.remove(this);
        break;
  }
  DrawImage(Board.m_Start.x + 87, Board.m_Start.y + 70);
}
/* Function used to fill all avalible moves into the Moves ArrayList
  Input: ArrayList MovesAvaliable that sent from CPU.
  Processing: Function running through all the avaliagle cells, that means that the next move is empty,
  adding them. Once no move avaliable moves, function starting to calcualte power of Human and power of,
  CPU, adding them too to the MovesAvaliable ArrayList.
```

```
 */
public void FillMovesAvaliable(ArrayList<Move> MovesAvaliable) {
    Cell[][] Mat = Parent.m_Board.ArCells;
    for (int i = 0; i < DirectionSize; i++) { // Running though all the directions.
        if (Mat[(m_Row + Board.Direction[i][1])][(m_Col + Board.Direction[i][0])] != null) { //
Checks only on actual cells.
            if (Mat[(m_Row + Board.Direction[i][1])][(m_Col + Board.Direction[i][0])].m_Type
== 1) { // Add to avaliable if empty cell.
                MovesAvaliable.add(new Move(new Point(m_Col, m_Row), i, 1, 0, false));
            } else { // If not empty - do the sizes calculation.
                /* Saving location. */
                int row = m_Row;
                int col = m_Col;
                /* Decleration two integers for power. */
                int humanPower = 0;
                int EnemyPower = 0;
                while ((Mat[row][col] != null) && (Mat[row][col].m_Type == m_Type)) { //
Adding to Human attack till same type and not null.
                    row += Board.Direction[i][1];
                    col += Board.Direction[i][0];
                    humanPower++;
                }
                if ((Mat[row][col] != null) && (humanPower <= 3)) { // If not stack ball and not
out.
                    if (Mat[row][col].m_Type == 1) {
                        MovesAvaliable.add(new Move(new Point(m_Col, m_Row), i, humanPower, 0,
false)); // Add free spots.
                    } else {
```

```
                    while ((Mat[row][col] != null) && (Mat[row][col].m_Type == 3)) { //
Consider enemy.
                        row += Board.Direction[i][1];
                        col += Board.Direction[i][0];
                        EnemyPower++;
                    }
                    if (humanPower > EnemyPower) { // Avaliable despite the power of enemy.
                        MovesAvaliable.add(new Move(new Point(m_Col, m_Row), i, humanPower,
EnemyPower, false));
                    }
                }
            }
        }
    }
  }
}
```

Player:

```
package abalonegame;
import java.util.ArrayList;


/* Abstract class Player that both Human and Computer extends from, which allows the
rating of the currect position.*/
public abstract class Player {
   /* Decleration of global values */
   GameFrame m_Parent;
   ArrayList<Cell> pieces;
```

```
    ArrayList<Spot> spotList;

    /* Player constractor */
    public Player(GameFrame Parent) {
        this.pieces = new ArrayList();
        this.spotList = new ArrayList();
        this.m_Parent = Parent;
    }

    /* Function that both players extend, but do differently. */
    public abstract void DoStep();

    /* Sets the pieces to be recognised as not visited */
    public void InitCells() {
        for (int i = 0; i < pieces.size(); i++) {
            ((Cell) pieces.get(i)).bolVisited = false;
        }
    }
    /* Function creating all spots for player */
    public void createSpots() {
        InitCells(); // Sets for all pieces Visited status to false.
        spotList.clear(); // Clearning previous created spots.
        Spot s = new Spot();
        spotList.add(s);
        makeSpot((Cell) pieces.get(0), ((Cell) pieces.get(0)).m_Type); // Function recursively
calculates the spot.
        ((Spot) spotList.get(0)).setGrade(); // Setting grade for the currect spot.
```

```
    for (int i = 1; i < pieces.size(); i++) {
        if (((Cell) pieces.get(i)).bolVisited == false) { /* Creating spot for unvisited spots */
            spotList.add(new Spot());
            makeSpot((Cell) pieces.get(i), ((Cell) pieces.get(i)).m_Type);
            ((Spot) spotList.get(spotList.size() - 1)).setGrade(); // Setting grade.
        }
    }
}

/* Function making the spots and adding them to the spot ArrayList
    Input; Cell and type.
    Processing; Function adding all cells to the currect spot according to the spots around it.
*/
private void makeSpot(Cell cell, int type) {
    if (cell == null) {
        return;
    }
    Cell[][] m = m_Parent.m_Board.ArCells; // New Cells array, based on real one.
    int row = cell.m_Row;
    int col = cell.m_Col;
    if ((cell.m_Type != type) || cell.bolVisited) { // Spot already made.
        return;
    }
    ((Spot) spotList.get(spotList.size() - 1)).listSpot.add(cell); // Else - Make the spot and
add it to the spots list.
    m[row][col].bolVisited = true;
    /* Making spot in all directions recursively. */
```

```
        makeSpot(m[Board.Direction[0][1] + row][Board.Direction[0][1] + col], type);
        makeSpot(m[Board.Direction[1][1] + row][Board.Direction[0][1] + col], type);
        makeSpot(m[Board.Direction[2][1] + row][Board.Direction[0][1] + col], type);
        makeSpot(m[Board.Direction[3][1] + row][Board.Direction[0][1] + col], type);
        makeSpot(m[Board.Direction[4][1] + row][Board.Direction[0][1] + col], type);
        makeSpot(m[Board.Direction[5][1] + row][Board.Direction[0][1] + col], type);
    }

    /* Function updating new cell in old cell */
    public void upDateList(Cell newCell, Cell oldCell)
    {
        pieces.remove(oldCell);
        pieces.add(newCell);
    }
}
```

Human:

```
package abalonegame;
public class HumanPlayer extends Player {
    /**
     * Creates a new instance of HumanPlayer
     */
    public HumanPlayer(GameFrame Parent) {
        super(Parent);
    }
```

```
    public void DoStep() { // Function DoStep is empty as the Human is the one who does the move.
    }
}
```

ComputerPlayer:

```
package abalonegame;
import java.util.ArrayList;

public class ComputerPlayer extends Player {
    ArrayList<Move> Avaliable; // All avaliable moves for CPU.
    /* ComputerPlayer constractor */
    public ComputerPlayer(GameFrame Parent) {
        super(Parent);
        this.Avaliable = new ArrayList();
    }

    /* Function making the CPU move after finding the best move.
        Input; Non.
        Processing; Making visible moves to find the best, and returns back for all avaliable moves,
    calculating Human moves as well. Eventually, best move being chose and made.
    */
    public void DoStep() {
        FillMoves(); // Setting up the all possible moves.
        Move BestMove = null; // Best move we're looking for.
        double MaxGrade = -30000.0D; // Minimal possible move.
        for (int i = 0; i < Avaliable.size(); i++) {
```

```
        ((Move) Avaliable.get(i)).DoMove(m_Parent); // Making a move for Humman in temp
board.
        pieces = m_Parent.m_Board.m_Comp.pieces; // Inits comp balls.
        ((Move) Avaliable.get(i)).grade = GetGrade(); // Calculationg of move grade.
        if (((Move) Avaliable.get(i)).pushOut) {
            ((Move) Avaliable.get(i)).grade += 100.0D; // High motivation to push out.
        }
        ((Move) Avaliable.get(i)).UndoMove(m_Parent);
        pieces = m_Parent.m_Board.m_Comp.pieces; // Sets back comp balls.
        if (((Move) Avaliable.get(i)).grade > MaxGrade) {
            MaxGrade = ((Move) Avaliable.get(i)).grade;
            BestMove = (Move) Avaliable.get(i);
        }
    }

    if (BestMove != null) {
        BestMove.virtual = false; // Not a virtual move and actually made one.
        BestMove.DoMove(m_Parent);
    }
    m_Parent.m_Board.ChangeTurn(); // Change turn.
  }

  /* Function that creates for all pieces it's avaliable moves and save it into ArrayList
'Avaliable' */
  private void FillMoves() {
    for (int i = 0; i < pieces.size(); i++) {
```

```
        ((Cell) pieces.get(i)).FillMovesAvaliable(Avaliable); // Fill all avaliable moves for
given pieces.
    }
  }

  /* Function to calcualte the grade of every spot
    Input; Non.
    Processing; Function clculating it's grade based on the spots,
  calculating Human's grade based on his Spots together, and setting best
  move to go on min-max calculation and to know best move.
  */
  private double GetGrade() {
    createSpots(); // Absract function that creates the spots of CPU.
    double grade = 0.0; // Decleration double for CPU grade.
    for (int i = 0; i < spotList.size(); i++) { // Calculate grade for every spot.
      grade += ((Spot) spotList.get(i)).SpotGrade;
    }
    grade /= spotList.size(); // Calculation recognizing the less spots, ..
                        // .. higher grade.
    /* Creating the spots for Human and calculating them. */
    m_Parent.m_Board.m_Human.createSpots();
    double HumGrade = 0.0; // Decleration double for Human grade.
    for (int i = 0; i < m_Parent.m_Board.m_Human.spotList.size(); i++) {
      HumGrade += ((Spot) m_Parent.m_Board.m_Human.spotList.get(i)).SpotGrade;
    }
    return ((m_Parent.m_Board.m_Human.spotList.size() - spotList.size()) * 4 + grade -
HumGrade); // Function calculating grade in min-max style.
```

```
    }
    /* Thread thinking to protect the moves to mix.
       Input: Non.
       Processing; sending to AI class that extends from Thread the ComputerPlayer.
    */
    void AIDoStep() {
       AI Ai = new AI(this);
       Ai.start();
    }
}
```

AI:

```
package abalonegame;
/* Artificial intelligence class to promise the moves won't be done at same time. */
public class AI extends Thread {
    ComputerPlayer Comp;
    /* AI Constractor */
    public AI(ComputerPlayer comp) {
       this.Comp = comp;
    }
    @Override
    public void run() {
       try {
          Thread.sleep(1000L); // Suspends the thread for 1000 miliseconds.
       } catch (InterruptedException ex) {
       }
       Comp.DoStep();
```

```
    }
}
```

Move:

```
package abalonegame;

import java.awt.Point;


public class Move {

    Point p;

    int dir, attack, enemyAttack;

    double grade;

    boolean pushOut, virtual;

    Board TempBoard = new Board();


    /* Move constractor */

    public Move(Point sour, int dir, int attack, int enemyAttack, boolean pushOut) {

        this.p = sour; // Source point.

        this.dir = dir; // Direcion.

        this.attack = attack; // Self-Attack power by size of balls selected.

        this.enemyAttack = enemyAttack; // Number of balls blocking the attack.

        this.pushOut = pushOut; // Boolean to tell if ball being pushed out.

        this.virtual = true; // Boolean if move isn't really being made, but for ..

        // .. the check of AI. Default option is 'true'.

    }


    /* Function managing a move in virtual board for CPU to think.

        Input; The GameFrame.
```

```
    Processing; Function creating a Temporary board, clearing from the frame the both
Selected and,
    unselected */
  public void DoMove(GameFrame g) {
    TempBoard.CopyBoard(g, g.m_Board.ArCells); // Copying currect board.

    /* Clearing balls */
    g.m_Board.SelectedBalls.clear();
    g.m_Board.Enemy.clear();

    Cell[][] mat = g.m_Board.ArCells; // New Cells array based on pieces.
    int row = p.y, col = p.x; // Having location of clicked point.s

    /* Adding both parties given in message sent giving us the powers. */
    for (int i = 0; i < attack; i++) {
      g.m_Board.SelectedBalls.add(mat[row][col]);
      row += Board.Direction[dir][1];
      col += Board.Direction[dir][0];
    }
    for (int i = 0; i < enemyAttack; i++) {
      g.m_Board.Enemy.add(mat[row][col]);
      row += Board.Direction[dir][1];
      col += Board.Direction[dir][0];
    }
    g.m_Board.MoveCells(dir, this); // Move being in a copied board.
  }
  /* Gets back to the old board.
```

```
        Input: GameFrame of the board.

        Processing; Copying to Parent, the preiou*/

    public void UndoMove(GameFrame Parent) {

        Parent.m_Board.CopyBoard(Parent, TempBoard.ArCells);

    }

}
```

Spot:

```
package abalonegame;

import java.util.ArrayList;


/* Being in spot and in the center - Grading system based in the fact that being in the center is fater

   then the borders. As long as your ball is futher from the borders, it's safer, and when more balls

   gathered in together in a pack there are more likely to be 3 row+col balls which cannot be moved.   */

public class Spot {

    public ArrayList<Cell> listSpot;

    public double SpotGrade;

    /* Spot constractor */

    public Spot() {

        this.listSpot = new ArrayList();

    }

    /* Function calculating SpotGrade

        Inputs; Non.

        Processing; Function running through all the spot nd calculating it's rows and cols, then it calculates,
```

```
it's grade depending of it's distance from the middle.*/
public void setGrade() {
    double dRow = 0.0,
           dCol = 0.0;
    /* Calculation of the spot */
    for (int i = 0; i < listSpot.size(); i++) {
        dRow += ((Cell) listSpot.get(i)).m_Row;
        dCol += ((Cell) listSpot.get(i)).m_Col;
    }

    dRow /= listSpot.size();
    dCol /= listSpot.size();

    double dDisCol = distanceColCal(((Cell) listSpot.get(0)).m_Col, dCol);
    double dMaxDis = Math.sqrt(Math.pow(dCol, 2) + Math.pow((((Cell)
listSpot.get(0)).m_Row - dRow), 2)); // Setting first Max.
    for (int i = 1; i < listSpot.size(); i++) {
        dDisCol = distanceColCal(((Cell) listSpot.get(i)).m_Col, dCol);
        double dis = Math.sqrt(Math.pow(dDisCol, 2) + Math.pow((((Cell)
listSpot.get(i)).m_Row - dRow), 2));
        if (dis > dMaxDis) {
            dMaxDis = dis;
        }
    }
    if (dMaxDis == 0.0) {
        dMaxDis = 1.0; // Setting minimal grade.
    }
```

```
        SpotGrade = listSpot.size() / dMaxDis;
        dDisCol = distanceColCal(10.0, dCol);
        double dCenterDis = Math.sqrt(Math.pow(dDisCol, 2) + Math.pow((5.0D - dRow), 2));
        if (dCenterDis == 0.0) {
            dCenterDis = 1.0;
        }
        SpotGrade += 2 * listSpot.size() / dCenterDis;
    }

    /* Calculating differently odds for cols.
        Input; Two doubles: Col of list, and col given to calculate.
        Processing; Function calculates the cols the way it can be compared right with the rows.
     */
    private double distanceColCal(double listCol, double dCol) {
        if (Math.abs(listCol - dCol) % 2 == 0) {
            return Math.abs(listCol - dCol) / 2.0D;
        }
        return Math.abs(listCol - dCol);
    }
}
```

GameFrame:

```
package abalonegame;
import java.awt.Graphics;
```

```
import java.awt.Image;
import java.awt.Point;
import javax.swing.ImageIcon;

public class GameFrame extends javax.swing.JFrame {
    /* Decleration of class values */
    public Board m_Board;
    int Dir;
    boolean Start;
    public static Point m_Start = new Point(100, 60);
    Image EmptyBoard = null, BarImage, Turn;
    public int wBoard, hBoard;
    /**
     * Creates new form GameFrame
     */
    public GameFrame() {
        initComponents();
        this.setLocationRelativeTo(null);
        this.setResizable(false);
        Start = false; // Setting to not started yet.

        /* Images for tools in Project Folder */
        EmptyBoard = getToolkit().createImage("Board.jpg");
        BarImage = getToolkit().createImage("Bar.png");
        Turn = getToolkit().createImage("Red.png");
```

```
    /* Images for Buttons in Project Folder */
    UpRightBut.setIcon(new ImageIcon("ru.png"));
    UpLeftBut.setIcon(new ImageIcon("lu.png"));
    RightBut.setIcon(new ImageIcon("r.png"));
    LeftBut.setIcon(new ImageIcon("l.png"));
    DownRightBut.setIcon(new ImageIcon("rd.png"));
    DownLeftBut.setIcon(new ImageIcon("ld.png"));
}

/* Starting game */
public void NewGame(int Num) {
    Start = true;
    Turn = getToolkit().createImage("Red.png"); // Red starts.
    m_Board = new Board(this, m_Start, Num); // Creating the board.
    repaint();
}

@Override
public void paint(Graphics g) {
    wBoard = 522;
    hBoard = 462;
    g.drawImage(EmptyBoard, m_Start.x, m_Start.y, wBoard, hBoard, this);

    int wBar = 241;
    int hBar = 39;
```

```
        g.drawImage(BarImage, m_Start.x + wBoard + 20, m_Start.y, wBar, hBar, this);
        g.drawImage(BarImage, m_Start.x + wBoard + 20, m_Start.y + 100, wBar, hBar, this);
        if (Start) {
            m_Board.Draw(g);
            g.drawImage(Turn, m_Start.x - 80, m_Start.y + 450, Cell.CellSizeW, Cell.CellSizeH,
this);
        }
        this.jMenuBar1.updateUI();
        this.jMenu1.updateUI();
        this.LeftBut.updateUI();
        this.RightBut.updateUI();
        this.DownLeftBut.updateUI();
        this.DownRightBut.updateUI();
        this.UpLeftBut.updateUI();
        this.UpRightBut.updateUI();
    }
    // <editor-fold defaultstate="collapsed" desc="Generated Code">
    private void initComponents() {
        NetWaitDialog = new javax.swing.JDialog();
        jLabel2 = new javax.swing.JLabel();
        jPanel1 = new javax.swing.JPanel();
        UpLeftBut = new javax.swing.JButton();
        UpRightBut = new javax.swing.JButton();
        LeftBut = new javax.swing.JButton();
        RightBut = new javax.swing.JButton();
        DownLeftBut = new javax.swing.JButton();
        DownRightBut = new javax.swing.JButton();
```

```
jMenuBar1 = new javax.swing.JMenuBar();
jMenu1 = new javax.swing.JMenu();
NewMenu = new javax.swing.JMenu();
PlayerMenuItem = new javax.swing.JMenuItem();
CompMenuItem = new javax.swing.JMenuItem();
jSeparator1 = new javax.swing.JSeparator();
ExitItem = new javax.swing.JMenuItem();
jMenu2 = new javax.swing.JMenu();
About = new javax.swing.JMenuItem();
NetWaitDialog.setModal(true);
jLabel2.setFont(new java.awt.Font("Arial", 1, 18)); // NOI18N
jLabel2.setText("Wait  to second  player");
org.jdesktop.layout.GroupLayout NetWaitDialogLayout = new
org.jdesktop.layout.GroupLayout(NetWaitDialog.getContentPane());
NetWaitDialog.getContentPane().setLayout(NetWaitDialogLayout);
NetWaitDialogLayout.setHorizontalGroup(

NetWaitDialogLayout.createParallelGroup(org.jdesktop.layout.GroupLayout.LEADING)
    .add(NetWaitDialogLayout.createSequentialGroup()
        .add(28, 28, 28)
        .add(jLabel2)
        .addContainerGap(25, Short.MAX_VALUE))
);
NetWaitDialogLayout.setVerticalGroup(

NetWaitDialogLayout.createParallelGroup(org.jdesktop.layout.GroupLayout.LEADING)
    .add(NetWaitDialogLayout.createSequentialGroup()
```

```
        .add(23, 23, 23)
        .add(jLabel2)
        .addContainerGap(19, Short.MAX_VALUE))
);
setDefaultCloseOperation(javax.swing.WindowConstants.EXIT_ON_CLOSE);
setBackground(new java.awt.Color(255, 255, 255));
addMouseListener(new java.awt.event.MouseAdapter() {
    public void mousePressed(java.awt.event.MouseEvent evt) {
        formMousePressed(evt);
    }
});
addComponentListener(new java.awt.event.ComponentAdapter() {
    public void componentResized(java.awt.event.ComponentEvent evt) {
        formComponentResized(evt);
    }
});
UpLeftBut.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        UpLeftButActionPerformed(evt);
    }
});
UpRightBut.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        UpRightButActionPerformed(evt);
    }
});
```

```
LeftBut.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        LeftButActionPerformed(evt);
    }
});
RightBut.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        RightButActionPerformed(evt);
    }
});
DownLeftBut.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        DownLeftButActionPerformed(evt);
    }
});
DownRightBut.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        DownRightButActionPerformed(evt);
    }
});
org.jdesktop.layout.GroupLayout jPanel1Layout = new
org.jdesktop.layout.GroupLayout(jPanel1);
jPanel1.setLayout(jPanel1Layout);
jPanel1Layout.setHorizontalGroup(
    jPanel1Layout.createParallelGroup(org.jdesktop.layout.GroupLayout.LEADING)
    .add(jPanel1Layout.createSequentialGroup()
```

```
.add(jPanel1Layout.createParallelGroup(org.jdesktop.layout.GroupLayout.LEADING)
            .add(jPanel1Layout.createSequentialGroup()
                .add(UpLeftBut, org.jdesktop.layout.GroupLayout.PREFERRED_SIZE, 25,
org.jdesktop.layout.GroupLayout.PREFERRED_SIZE)
                .addPreferredGap(org.jdesktop.layout.LayoutStyle.RELATED)
                .add(UpRightBut, org.jdesktop.layout.GroupLayout.PREFERRED_SIZE, 25,
org.jdesktop.layout.GroupLayout.PREFERRED_SIZE))
            .add(jPanel1Layout.createSequentialGroup()
                .add(LeftBut, org.jdesktop.layout.GroupLayout.PREFERRED_SIZE, 25,
org.jdesktop.layout.GroupLayout.PREFERRED_SIZE)
                .addPreferredGap(org.jdesktop.layout.LayoutStyle.RELATED)
                .add(RightBut, org.jdesktop.layout.GroupLayout.PREFERRED_SIZE, 25,
org.jdesktop.layout.GroupLayout.PREFERRED_SIZE))
            .add(jPanel1Layout.createSequentialGroup()
                .add(DownLeftBut, org.jdesktop.layout.GroupLayout.PREFERRED_SIZE, 25,
org.jdesktop.layout.GroupLayout.PREFERRED_SIZE)
                .addPreferredGap(org.jdesktop.layout.LayoutStyle.RELATED)
                .add(DownRightBut, org.jdesktop.layout.GroupLayout.PREFERRED_SIZE, 25,
org.jdesktop.layout.GroupLayout.PREFERRED_SIZE)))
        .addContainerGap(68, Short.MAX_VALUE))
    );
    jPanel1Layout.setVerticalGroup(
      jPanel1Layout.createParallelGroup(org.jdesktop.layout.GroupLayout.LEADING)
      .add(jPanel1Layout.createSequentialGroup()
        .add(90, 90, 90)

.add(jPanel1Layout.createParallelGroup(org.jdesktop.layout.GroupLayout.BASELINE)
            .add(UpLeftBut, org.jdesktop.layout.GroupLayout.PREFERRED_SIZE, 20,
org.jdesktop.layout.GroupLayout.PREFERRED_SIZE)
```

```
                .add(UpRightBut, org.jdesktop.layout.GroupLayout.PREFERRED_SIZE, 20,
org.jdesktop.layout.GroupLayout.PREFERRED_SIZE))

            .addPreferredGap(org.jdesktop.layout.LayoutStyle.RELATED)


.add(jPanel1Layout.createParallelGroup(org.jdesktop.layout.GroupLayout.BASELINE)

                .add(LeftBut, org.jdesktop.layout.GroupLayout.PREFERRED_SIZE, 20,
org.jdesktop.layout.GroupLayout.PREFERRED_SIZE)

                .add(RightBut, org.jdesktop.layout.GroupLayout.PREFERRED_SIZE, 20,
org.jdesktop.layout.GroupLayout.PREFERRED_SIZE))

            .addPreferredGap(org.jdesktop.layout.LayoutStyle.RELATED,
org.jdesktop.layout.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)

        .add(jPanel1Layout.createParallelGroup(org.jdesktop.layout.GroupLayout.BASELINE)

                .add(DownLeftBut, org.jdesktop.layout.GroupLayout.PREFERRED_SIZE, 20,
org.jdesktop.layout.GroupLayout.PREFERRED_SIZE)

                .add(DownRightBut, org.jdesktop.layout.GroupLayout.PREFERRED_SIZE, 20,
org.jdesktop.layout.GroupLayout.PREFERRED_SIZE))

            .addContainerGap())

    );

    jMenu1.setText("Game");

    NewMenu.setIcon(new
javax.swing.ImageIcon("C:\\Users\\borod\\OneDrive\\Documents\\NetBeansProjects\\A
balone\\Selected.JPG")); // NOI18N

    NewMenu.setText("New");

    PlayerMenuItem.setText("vs Player");

    PlayerMenuItem.addActionListener(new java.awt.event.ActionListener() {

        public void actionPerformed(java.awt.event.ActionEvent evt) {

            PlayerMenuItemActionPerformed(evt);

        }

    });

    NewMenu.add(PlayerMenuItem);
```

```
        CompMenuItem.setText("vs Computer");
        CompMenuItem.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                CompMenuItemActionPerformed(evt);
            }
        });
        NewMenu.add(CompMenuItem);
        jMenu1.add(NewMenu);
        jMenu1.add(jSeparator1);

ExitItem.setAccelerator(javax.swing.KeyStroke.getKeyStroke(java.awt.event.KeyEvent.VK_E
SCAPE, 0));
        ExitItem.setIcon(new
javax.swing.ImageIcon(getClass().getResource("/abalonegame/Black.JPG"))); // NOI18N
        ExitItem.setText("Exit");
        ExitItem.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                ExitItemActionPerformed(evt);
            }
        });
        jMenu1.add(ExitItem);
        jMenuBar1.add(jMenu1);
        jMenu2.setText("Help");
    About.setAccelerator(javax.swing.KeyStroke.getKeyStroke(java.awt.event.KeyEvent.VK_H,
0));
        About.setText("About");
        About.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
```

```
            AboutActionPerformed(evt);
        }
    });
    jMenu2.add(About);
    jMenuBar1.add(jMenu2);
    setJMenuBar(jMenuBar1);
    org.jdesktop.layout.GroupLayout layout = new
org.jdesktop.layout.GroupLayout(getContentPane());
    getContentPane().setLayout(layout);
    layout.setHorizontalGroup(
        layout.createParallelGroup(org.jdesktop.layout.GroupLayout.LEADING)
        .add(layout.createSequentialGroup()
            .addContainerGap()
            .add(jPanel1, org.jdesktop.layout.GroupLayout.PREFERRED_SIZE,
org.jdesktop.layout.GroupLayout.DEFAULT_SIZE,
org.jdesktop.layout.GroupLayout.PREFERRED_SIZE)
            .addContainerGap(787, Short.MAX_VALUE))
    );
    layout.setVerticalGroup(
        layout.createParallelGroup(org.jdesktop.layout.GroupLayout.LEADING)
        .add(layout.createSequentialGroup()
            .add(85, 85, 85)
            .add(jPanel1, org.jdesktop.layout.GroupLayout.PREFERRED_SIZE,
org.jdesktop.layout.GroupLayout.DEFAULT_SIZE,
org.jdesktop.layout.GroupLayout.PREFERRED_SIZE)
            .addContainerGap(239, Short.MAX_VALUE))
    );
    pack();
```

```
}// </editor-fold>
private void CompMenuItemActionPerformed(java.awt.event.ActionEvent evt) {
    NewGame(1);
    repaint();
}
private void PlayerMenuItemActionPerformed(java.awt.event.ActionEvent evt) {
    NewGame(0);
    repaint();
}
/* Moves Buttons */
private void DownRightButActionPerformed(java.awt.event.ActionEvent evt) {
    Dir = 3;
    m_Board.DoStep(Dir);
}
private void DownLeftButActionPerformed(java.awt.event.ActionEvent evt) {
    Dir = 4;
    m_Board.DoStep(Dir);
}
private void RightButActionPerformed(java.awt.event.ActionEvent evt) {
    Dir = 2;
    m_Board.DoStep(Dir);
}
private void LeftButActionPerformed(java.awt.event.ActionEvent evt) {
    Dir = 5;
    m_Board.DoStep(Dir);
}
```

```
private void UpRightButActionPerformed(java.awt.event.ActionEvent evt) {
    Dir = 1;
    m_Board.DoStep(Dir);
}
private void UpLeftButActionPerformed(java.awt.event.ActionEvent evt) {
    Dir = 0;
    m_Board.DoStep(Dir);
}
private void formMousePressed(java.awt.event.MouseEvent evt) {
    if (m_Board != null) {
        m_Board.Click(evt.getPoint());
        repaint();
    }
}
private void ExitItemActionPerformed(java.awt.event.ActionEvent evt) {
    System.exit(0);
}
private void formComponentResized(java.awt.event.ComponentEvent evt) {
    repaint();
}

private void AboutActionPerformed(java.awt.event.ActionEvent evt) {
    AboutFrame f = new AboutFrame();
    f.setVisible(true);
}
/**
```

```
 * @param args the command line arguments
 */
public static void main(String args[]) {
    java.awt.EventQueue.invokeLater(new Runnable() {
        public void run() {
            new GameFrame().setVisible(true);
        }
    });
}
// Variables declaration - do not modify
private javax.swing.JMenuItem About;
private javax.swing.JMenuItem CompMenuItem;
private javax.swing.JButton DownLeftBut;
private javax.swing.JButton DownRightBut;
private javax.swing.JMenuItem ExitItem;
private javax.swing.JButton LeftBut;
public javax.swing.JDialog NetWaitDialog;
private javax.swing.JMenu NewMenu;
private javax.swing.JMenuItem PlayerMenuItem;
private javax.swing.JButton RightBut;
private javax.swing.JButton UpLeftBut;
private javax.swing.JButton UpRightBut;
private javax.swing.JLabel jLabel2;
private javax.swing.JMenu jMenu1;
private javax.swing.JMenu jMenu2;
private javax.swing.JMenuBar jMenuBar1;
```

```
    private javax.swing.JPanel jPanel1;
    private javax.swing.JSeparator jSeparator1;
    // End of variables declaration
}
```

There is another Frame for rules explanation.